ASIC Design Flow

From RTL to Physical Chip Design

Nariman Mani | Classroom tutorial | October 2026

How does a description of what a digital circuit should do become a physical chip that works correctly and meets timing?

Start with one four-bit counter: **store** a number, **calculate** its next value, and **update** it at a clock edge. Follow that example through RTL, synthesis, timing, placement, clock distribution, routing, and the checks needed before layout handoff.

Use with ASIC Explorer. Predict a result, try the guided experiment, observe the circuit or waveform, then explain why it happened. The opening primer uses plain language; later sections provide worked calculations and reference material.

Teaching route. The suggested core takes 36 minutes, with four minutes reserved for questions and recovery. Optional sections support later practice. These are planning windows, not a measured delivery time.

Scope. The app provides actual Yosys synthesis and DigitalJS functional simulation. Its physical-design and timing interactions are independent illustrative teaching models, without a foundry PDK or fabrication-ready output.

Companion source: github.com/narimanmani/ASIC-Explorer

Start here: the hardware picture in plain language

One example for the whole lesson. Imagine a small scoreboard that remembers a number. An update signal tells it when it may change. If counting is enabled, it adds one; otherwise, it keeps its number. Reset tells it to show zero at the next update. Our scoreboard has four binary digits, so it can show 0 through 15. After 15 it wraps to 0. This is the event counter we will use throughout.

The three jobs: store, calculate, update

Store. A *register* remembers the current number. One storage bit is commonly implemented by a flip-flop; four bits require four storage bits. In the app, a single DFF symbol can represent this four-bit group.

Calculate. *Logic* works out the next number. An adder calculates current count plus one. Selection logic chooses zero, the incremented number, or the current number according to reset and enable.

Update. A *clock* is a repeating electrical signal. Its rising edge, the transition from 0 to 1, is the update event for this counter. The register accepts the selected value at that edge and holds it between edges.

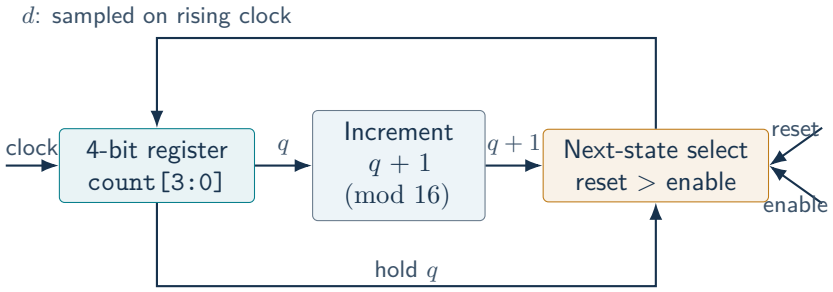


Figure 1: Read the picture as three jobs: the register stores, the logic calculates and selects, and the clock edge permits an update. Follow the feedback arrow from current count to the adder.

Read the symbols. `Q` is the value currently stored. `D` is the candidate value entering storage. The little triangle marks an edge-sensitive clock input. A box marked `+` calculates a sum. A mux, short for multiplexer, chooses one of several values. A line between symbols carries a signal; a multi-bit line carries several bits together. A schematic shows connections, not chip positions.

App practice: Your first two-minute experiment

1. Open **RTL & behavior**, choose **Guided view**, select **4-bit event counter**, and choose **Word level**. Restart with the circular-arrow control.
2. Predict two enabled updates from zero. Click **Step clock** twice: the number becomes 2, with binary digits `0010`.
3. Turn **Enable** off and step again: the number stays 2. The clock advanced, but the register kept its number.
4. Turn **Reset** on. Look before stepping: the number is still 2. Step once: it becomes 0. This reset takes effect at an edge.

Say it back. "The register remembers the number. Logic calculates what comes next. A clock edge updates it when the control rules allow."

The bridge from software to hardware

You can understand the counter's rule as a software-style state update: "At an update event, if reset is on, store zero; otherwise, if enable is on, store current value plus one." This is a useful way to predict values. **Its limit:** the resulting circuit contains physical logic that responds continuously to signals. It does not execute all the source statements as a CPU instruction sequence. The register waits for its clock edge while the logic can already be calculating.

RTL means register-transfer level. It describes stored values and the calculations used to update them. In our SystemVerilog example, `always_ff @(posedge clk)` names the update event, `if (rst)` expresses reset priority, and `count <= count + 4'd1` gives the next-value rule. Here `<=` is a nonblocking assignment to hardware state, rather than a "less than or equal" comparison. `4'd1` means the decimal number 1 represented with four bits. Section 3 contains the complete module.

Synthesis is a hardware compilation step. The tool turns these rules into components and their connections. The resulting *netlist* is like a component inventory plus a wiring list. An operation such as addition can expand into many small gates. A *standard cell* is a predesigned physical building block whose behavior, dimensions, timing, and power are supplied by a technology library. The app shows real synthesis into generic components; foundry-library selection belongs to a subsequent implementation flow.

App practice: Look inside the calculation

1. Open **Gates & mapping**, select the prepared counter, restart, and choose **Word level**. Click the adder to see its explanation and associated source when available.
2. Step twice. Switch to **Gate level** and step once more. The count becomes 3, although the drawing has smaller components.
3. Explain why a different drawing can implement the same rule. For optional programming practice, change `4'd1` to `4'd2`, click **Synthesize**, and check 0, 2, 4. Restore the prepared counter afterward.

Timing: a deadline and a protection window

Logic and wires take time to deliver a new value. **Setup** asks whether that value settles early enough before the next receiving clock edge. Think "ready before the next update." In the app's independent path model, the latest arrival is 1.25 ns and the deadline is 1.55 ns, leaving $1.55 - 1.25 = 0.30$ ns to spare. One nanosecond is one billionth of a second. We call this remaining time *slack*. Positive slack means the modeled requirement is met; negative slack means it is missed.

Hold asks whether the old value remains stable briefly after the current receiving edge. Think "do not change what was just sampled too soon." A slower clock can give the *next* update more time, but it does not lengthen this protection window around the *current* edge. A short path may need added delay. The full equations, clock-skew convention, and margins are explained later; first understand these two different questions.

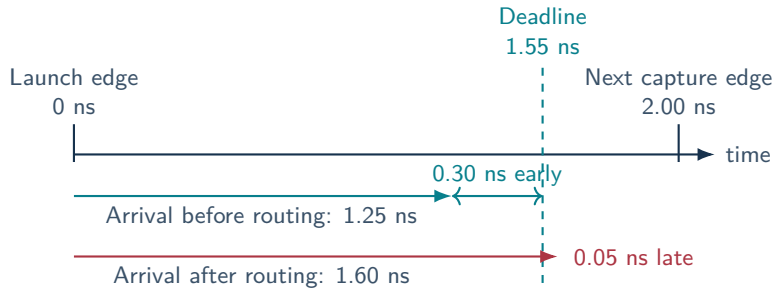


Figure 2: Keep the deadline fixed and compare the two arrival times. The wire change consumes the original setup margin. Use the full timing equations later.

App practice: Read the timing story before the equations

Controls for this activity. If a step changes a slider, click **Show equations and sliders** in Guided view first.

1. Open **Timing** in **Guided view**. Choose **Before routing**. Read arrival 1.25 ns, deadline 1.55 ns, and setup slack +0.30 ns.
2. Predict the effect of a slower wire. Choose **After routing**: arrival is 1.60 ns, so setup slack is –0.05 ns. The deadline is unchanged.
3. Choose **Slower clock**. Setup improves, but hold stays –0.04 ns. Choose **Repair hold**: hold becomes +0.02 ns and setup becomes +0.24 ns.

Each preset begins an independent case. The Repair hold result is not a repair applied to the After routing case. This teaching model is separate from the editable counter.

Physical design: make a buildable arrangement

Floorplan means reserve the chip's space, including room for power and major blocks. **Placement** means give each cell a location. Connected cells that are far apart can need longer wires. **Clock-tree synthesis** builds a distribution network for the clock, which also takes time to travel. **Routing** builds the signal wires in metal layers, joined by *vias*. A via is like an elevator joining floors, although a chip via is a conducting connection between layers. **Extraction** estimates wire resistance and capacitance from the geometry so timing can use a more realistic wire model. A long wire is an electrical element, not a free connection.

App practice: Connect locations to timing

1. In **Floorplan & place**, click **Reset layout**: connection length is 1020 teaching units. Click **Compact placement**: it becomes 455. The wiring relationships stay the same.
2. In **Clock tree**, reset and read the four arrival labels. Click **Build balanced clock tree**: the illustrative spread changes from 0.30 ns to 0.00 ns.
3. In **Routing & RC**, turn M2 off and on. Find the small via squares where metal layers meet. Click **Reveal parasitic RC**: `R` and `C` illustrate resistance and capacitance.

These views use an independent six-cell teaching illustration. The clock arrivals are assigned examples, and the RC overlay is symbolic. No physical-design or extraction tools run in these screens.

Approval: several checks, then manufacturing

A behavior test checks whether the counter follows its rules. Timing analysis checks deadlines and protection windows. **DRC**, design rule checking, checks geometry such as minimum wire spacing. **LVS**, layout versus schematic, checks whether devices and connections extracted from the layout match the reference circuit. A repair must pass the relevant checks again. **Tapeout** hands approved layout data to the manufacturing flow; fabrication, packaging, testing, and validation still follow.

App practice: Predict the check that catches the mistake

1. In **Signoff**, choose **Spacing violation**. DRC fails and LVS passes in this prepared case.
2. Choose **Broken connection**. LVS fails and DRC passes. Finish with **Clean layout**.
3. Explain aloud: "Behavior asks whether it counts correctly; timing asks whether values arrive at the right time; physical checks ask whether the layout follows the rules and implements the intended connections."

Before studying the details, check yourself. Explain why the number can stay 2 while the clock continues; why a longer wire can cause a timing failure without a code change; and why a layout can pass a spacing check but contain a broken connection. If one answer is unclear, repeat that experiment before adding more terminology.

Teaching route and learning outcomes

By the end of the lecture, students should be able to explain the counter using storage, calculation, and clocked updates; follow its rules through synthesis and physical design; read a simple timing margin; and choose which check investigates a particular failure.

Elapsed time	Core teaching activity
0–2 min	Introduce the counter and the three correctness questions.
2–7 min	Store, calculate, update; read a waveform and the short RTL rule.
7–9 min	App: two increments, disable, then synchronous reset.
9–11:30 min	Behavior checks and synthesis as hardware compilation.
11:30–12:30 min	App: adder, word-level view, then gate-level view.
12:30–15 min	Revisit the flow map; explain the path from one register to another.
15–18 min	Setup as a deadline; hold as protection of the current sample.
18–20 min	App: before/after routing, slower clock, and hold repair.
20–22:30 min	Timing requirements, floorplan, and placement.
22:30–23:30 min	App: compare reset and compact placement.
23:30–26 min	Clock distribution, metal layers, and wire electrical effects.
26–27:30 min	App: clock-tree illustration, then metal layers and RC overlay.
27:30–30 min	Brief power/test context; explain the independent signoff checks.
30–31 min	App: spacing violation, broken connection, and clean layout.
31–33:30 min	Repair and recheck; tapeout and manufacturing.
33:30–36 min	Student diagnosis and a plain-language recap.
36–40 min	Question and interruption buffer; no new core material.

Preparation and delivery

The beginner slide deck follows this route. Speaker notes contain only words to say. Its separate `App_Demo_Cues.txt` lists the exact screen actions and expected observations. Rehearse the first-pass explanation before the detailed sections. Use **Guided view** for the live demonstrations. If questions consume the buffer, omit the 26–27:30 clock/routing demonstration and explain it with the figures; preserve the counter and timing experiments. The app displays the same demo windows in Teaching notes. Stop introducing material at minute 36 and conclude by minute 40.

Teaching design. This organization connects unfamiliar concepts to prior knowledge, introduces a concrete example before equations, and revisits the main questions. Examples include explicit limits to prevent the analogy becoming a misconception; see Carnegie Mellon's lecture guidance [1]. These are design choices, not a measured claim of learning effectiveness.

Using ASIC Explorer for practice

The companion. ASIC Explorer is the Next.js teaching app accompanying this tutorial. Its editable HDL is synthesized by Yosys through YoWASP, converted by `yosys2digitaljs`, and functionally simulated by DigitalJS [12, 13, 14]. Circuit symbols and signal values come from those netlists. Timing, placement, clock distribution, routing, and signoff screens are separate teaching models; editing RTL does not regenerate a physical layout or its timing.

Start on the computer where you will practice

1. Obtain `web-app/ASIC_Explorer_App.zip` from the accompanying teaching package or Overleaf project files. Extract its `asic-teaching-app` folder. The app source is also the sibling `asic-teaching-app` folder in the prepared local workspace.
2. Use Node.js 22.19 or later. In the extracted folder, double-click `Start-ASIC-Explorer.cmd` on Windows. Alternatively, open a terminal in that folder and run `npm ci` once, then `npm run build` and `npm start`. Use the production build for the lecture and keep its terminal running.
3. Open http://127.0.0.1:4175. This address means *your own computer*; every student needs a local running copy for independent practice. Overleaf stores the tutorial and source bundle; it does not host the running Next.js app. A teacher can demonstrate one local copy through screen sharing.

Practice routine. Predict, perform the numbered steps, record the observation, then explain it using the section's concept. Unless a box specifies otherwise, select the named example and use the circular-arrow **Restart simulation** control to begin at a known state. For the supplied counters, restart applies an initialization reset edge, clears the visible trace, then sets `rst=0`, `en=1`, and `count=0`. This explains why the app starts at 0 while Figure 6 illustrates unspecified startup before reset. The visible clock-edge count excludes that initialization edge.

Guided view and Full lab. The app opens in Guided view with one practice step at a time. Predict, try the action, then use **Show explanation**; **Next practice** advances the instruction, without changing the circuit. Full lab shows the additional prediction question. Timing initially shows arrival, deadline, and slack; click **Show equations and sliders** for any exercise below that changes a slider. Switching learning views preserves the current model.

Controls. Step clock performs one rising and falling clock cycle. **Reset** is an input switch, while **Restart simulation** reinitializes the experiment. **Word level** and **Gate level** show two compiled abstractions and retain their respective simulation state while you switch views. After an edit, click **Synthesize**; the diagram continues to show the last successful source and live controls stay disabled until source and circuit agree. Selecting a prepared example reloads its compiled circuit; synthesizing new source restarts its simulation. Select a component to inspect compiler-supplied source associations. Some optimized components have no useful source position.

Navigation and evidence. Use the eight lesson buttons at left; the links below open their local lesson addresses. **P** toggles presentation mode, **N** opens teaching notes, **R** restarts simulation, and **Space** steps the clock when focus is outside the editor and form inputs. Focused placement cells use arrow keys to move. Zoom/fit the circuit for legibility. Save the circuit with **Download circuit SVG**, the trace with **Export waveform CSV**, and the source with **Download HDL source**. The trace uses logical event steps and settled values, not silicon delay in ns; X denotes an unknown value. **Teaching notes** links each lesson to an original diagram and its vector PDF.

App lesson	Sections and practice topics
01 Design intent	Flow, requirements, and the counter contract.
02 RTL & behavior	State, enable, reset, wraparound, and functional checks.
03 Gates & mapping	Real synthesis, source inspection, generic gates, and artifact roles.
04 Timing	Setup, hold, skew, timing budgets, and closure.
05 Floorplan & place	Illustrative utilization, cell movement, and wire length.
06 Clock tree	Illustrative sink arrivals and their timing implications.
07 Routing & RC	Metal, vias, and conceptual parasitics.
08 Signoff	Toy spacing/connectivity cases and missing handoff evidence.

Fit practice around the lecture. The route above, the beginner slide deck, and the app's Teaching notes use the same elapsed times. Live demonstrations consume 8.5 minutes inside the 36-minute route. The numbered practice boxes below are for independent preparation or later student work, rather than additional live demonstrations. For a first pass, read each section's *Plain-language starting point*, try its activity, and return to the detailed text when the observation makes sense. Presets select fresh timing cases; physical screens can reinitialize when you navigate away, so use the stated starting control.

Contents

Start here: the hardware picture in plain language . . . 1
Teaching route and learning outcomes . . . 5
Using ASIC Explorer for practice . . . 6
1 The map: from an idea to a physical implementation . . . 8
2 Specify a small design before writing RTL . . . 10
3 RTL describes hardware that operates concurrently . . . 11
4 Verify behavior and preserve it through synthesis . . . 12
5 Synthesis chooses an implementation from a library . . . 14
6 Technology data connects logical and physical tools . . . 15
7 Timing: two registers and the path between them . . . 16
8 Setup: calculate the latest allowed arrival . . . 17
9 Hold: protect the old value at the current edge . . . 19
10 Constraints express the timing contract . . . 20
11 Design for test: access to manufactured internal state . . . 22
12 Floorplanning: reserve a feasible physical home . . . 23
13 Placement: locations alter electrical behavior . . . 24
14 Clock tree synthesis: deliver edges to the registers . . . 25
15 Routing: build wires and vias that obey the technology . . . 26
16 Extraction: replace wire guesses with physical models . . . 28
17 Power and reliability constrain implementation . . . 29
18 Signoff: several independent questions . . . 30
19 Closure: repairs must survive the checks . . . 31
20 Tapeout is the handoff; manufacturing creates the chip . . . 33
21 Closing exercise and retrieval recap . . . 33
A Preparation questions and concise answers . . . 34
B Student practice with solutions . . . 36
C A reproducible lab extension and report-reading guide . . . 37
D Reusable visual catalog . . . 38
E Glossary . . . 39
Source notes and further reading . . . 41

1 The map: from an idea to a physical implementation

Lecture window: 12:30–13:30 minutes. *Start with the whole journey; return to the map at transitions.*

Plain-language starting point. A chip starts with rules for what it should do. We turn those rules into connected components, then assign positions and build wires. At each stage, we gain a more detailed representation and check that the intended behavior survives.

An **application-specific integrated circuit (ASIC)** serves a particular application. We focus on **synchronous digital standard-cell design**; larger chips can also contain memory macros and analog/IP blocks.

Register-transfer level (RTL) describes stored state and the combinational computations that determine its next value. **Synthesis** transforms synthesizable RTL into a technology-mapped netlist. A **netlist** names cell instances and the nets connecting their pins. **Physical design** assigns locations and constructs actual interconnect geometry. Tapeout commonly includes a GDSII or OASIS layout and required supporting files and reports.

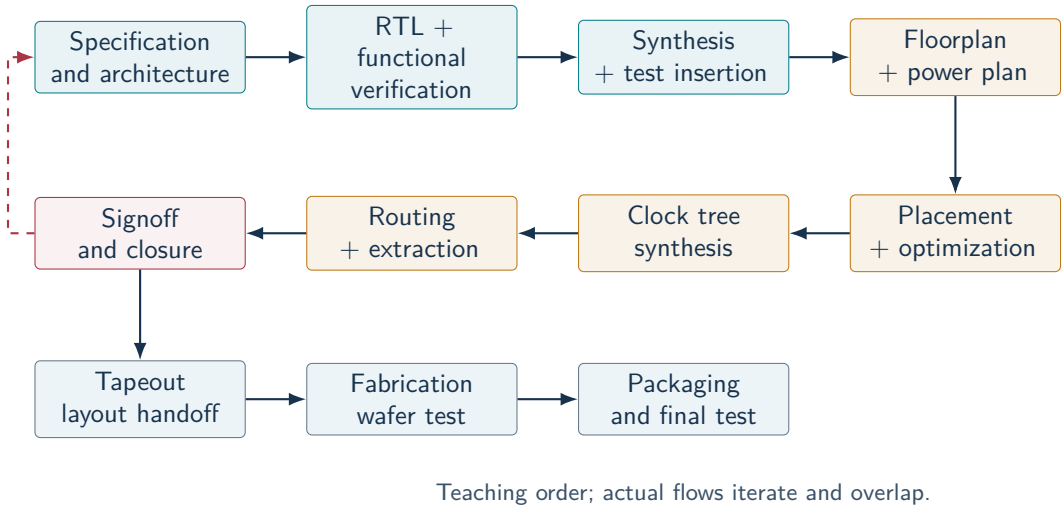


Figure 3: The end-to-end flow, with a closure loop. Test planning and verification span multiple stages.

The sequence is iterative: post-route timing failure may require new placement, sizing, or architecture. An RTL change triggers appropriate verification and downstream checks. **Verification is continuous.**

Front-end work usually covers architecture through synthesis; back-end work covers implementation/signoff. DFT and timing span both. OpenROAD provides a concrete educational flow [2].

Transition. "At each step we describe the same intended circuit with more detail. What changes, and what must remain true?"

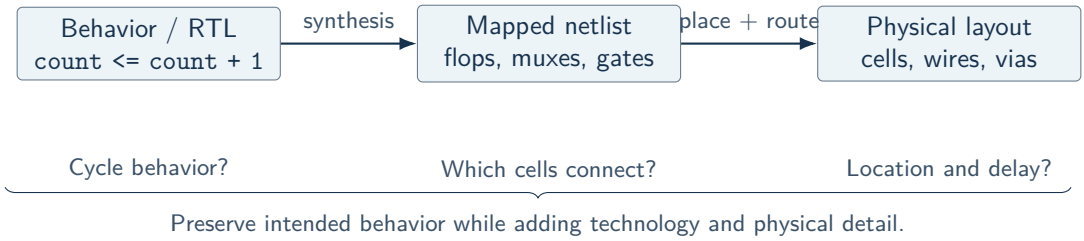


Figure 4: Three representations: behavior, connectivity, and geometry.

App practice: Find the three representations

1. Open **Design intent (01)**. Read Store, Calculate, and Update, then the Behavior, State, Timing goal, and Implementation descriptions. State the counter's contract in one sentence.
2. Click **Explore the RTL and circuit**. Select **4-bit event counter**, restart, and choose **Word level**. Identify the HDL rule, the adder, and the four-bit DFF.
3. Open **Floorplan & place (05)**. Identify the die, core, rows, and cells; explain what geometry adds to a connectivity diagram.

Expected observation. The RTL view shows connected hardware; the placement view introduces locations and space. **Explain.** These are different abstraction levels. The six-cell physical illustration is an independent teaching model, not a placement generated from this counter's netlist.

2 Specify a small design before writing RTL

Lecture window: 2–4 minutes. *Make requirements concrete enough to test.*

Plain-language starting point. Write the counter rule before the code: reset to zero, count when enabled, otherwise keep the number. The specification is the answer key for our tests.

Our running example is a **4-bit enabled event counter**, a tiny building block that could be used in sensor monitoring. We assume its inputs already belong to the clock domain. Counting an asynchronous external event would need additional interface design.

Requirement	Exact meaning
State	`count[3:0]` is an unsigned value from 0 to 15.
Clock	State changes on the rising edge of `clk`.
Reset	Active-high, synchronous `rst`; zero at a rising edge.
Enable	If `en=1` and reset is low, increment once per edge.
Hold	If reset and enable are both low, retain the state.
Priority	Reset wins when reset and enable are both high.
Overflow	Arithmetic wraps modulo 16: 15 becomes 0.
Interface	`rst` and `en` meet clock-domain timing assumptions.
Timing target	Illustrative 2.00 ns period (500 MHz), not measured capability.

$$q_{next} = \begin{cases} 0, & rst = 1, \\ (q+1) \bmod 16, & rst = 0 \text{ and } en = 1, \\ q, & rst = 0 \text{ and } en = 0. \end{cases}$$

A real specification also resolves interface budgets, operating conditions, power expectations, startup/reset behavior, and test requirements. The toy specification fixes the behavior and assumptions needed for this lesson.

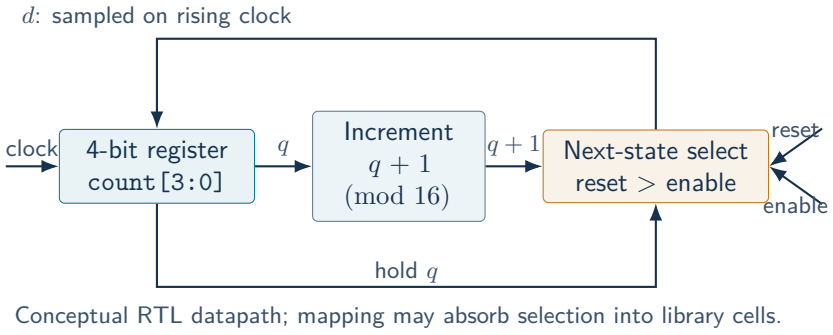


Figure 5: Conceptual next-state datapath. The register breaks the feedback path; this is not a combinational loop.

Prediction question 1

If `count=15`, `en=1`, and `rst=0`, what follows the next rising edge? **Answer: 0.** If both reset and enable are high, it is also 0 because reset has priority.

App practice: Test the reset-priority contract

1. Open **RTL & behavior (02)**, select **4-bit event counter**, and restart. Confirm `count=0`, `Reset=0`, and `Enable=1`.
2. Click **Step clock** twice. Predict and record the decimal value 2 and binary value `0010`.
3. Turn **Reset** on while **Enable** remains on. Check the output before stepping: it remains 2 because reset is synchronous.
4. Step once. The value becomes 0. Explain which requirement determines the result when reset and enable are both asserted.

Expected observation. Reset wins at the active edge. **Explain.** A testable specification resolves both input priority and the instant at which the state may change. Restart before beginning the next independent activity.

3 RTL describes hardware that operates concurrently

Lecture window: 4–9 minutes. *Read the code as a circuit, then predict a waveform.*

Plain-language starting point. The code names an update event and a next-value rule. The adder can calculate while the register still holds the old number. Only the chosen clock edge updates the stored value.

Listing 1: Synthesizable SystemVerilog for the running counter.

```
module event_counter (
  input  logic       clk,
  input  logic       rst,
  input  logic       en,
  output logic [3:0] count
);
  always_ff @(posedge clk) begin
    if (rst)
      count <= 4'd0;
    else if (en)
      count <= count + 4'd1;
  end
endmodule
```

`always_ff` expresses sequential intent. A nonblocking assignment `<=` evaluates the right-hand side at the triggering edge and schedules the update. Omitting the final `else` in this *clocked* process means the register holds its state; it does **not** infer a latch. In an incompletely assigned *combinational* process, retaining a value can infer a latch instead.

The four-bit width deliberately discards overflow. Hardware is concurrent: combinational increment/selection logic responds to inputs, while the register updates on edges. Do not read the HDL as a software loop that executes one statement at a time across the chip.

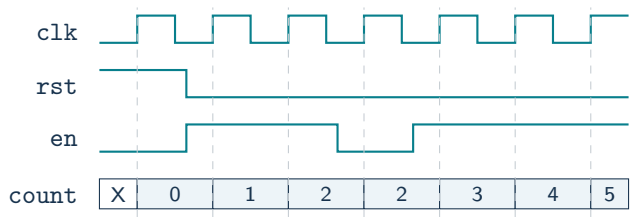


Figure 6: Predicted RTL behavior. X is unspecified startup state until synchronous reset is sampled. Real cells have clock-to-Q delay.

Reset is sampled at the first edge, two increments follow, a disabled cycle holds 2, and counting resumes. The diagram shows values after ideal RTL updates, not an analog or measured timing trace. Synchronous reset does not change state immediately when the reset input changes.

Check the hold cycle

"Why is the value 2 at two consecutive edges?" Enable is low at the second edge, so the flip-flops retain state. This connects the specification, code, and waveform.

Preparation depth. Check widths, signedness, reset choices, complete combinational assignments, and multiple drivers. Simulation-only delays, arbitrary file I/O, and general testbench behavior are not ordinary synthesizable hardware. Tool support for SystemVerilog constructs must be confirmed. Startup initialization and a reset signal are separate design decisions.

App practice: Build a waveform from predictions

1. In **RTL & behavior (02)**, select **4-bit event counter** and restart. Read the `always_ff` block before touching the controls.
2. Step twice: `count` becomes 1, then 2. Turn **Enable** off and step again: the clock toggles, but count stays 2.
3. Turn **Enable** on and step: count becomes 3. Compare the count row with the rising-edge markers in **Waveforms**.
4. Assert **Reset** between edges, observe the held value, then step and observe 0. Export the trace with **Export waveform CSV**.
5. Select **Counter · asynchronous reset**, restart, and step once. Assert Reset without stepping: count becomes 0 immediately. Find `posedge rst` in the source.

Explain. The event control determines when state can change. The trace has logical rising/falling samples; it does not measure propagation delay or asynchronous-reset recovery/removal timing.

4 Verify behavior and preserve it through synthesis

Lecture window: 9–10 minutes. *Separate behavioral evidence from timing evidence.*

Plain-language starting point. A behavior test is like a software test: choose inputs, predict the result, and compare. After hardware compilation, also check that the resulting circuit preserves the rule. Passing examples alone cannot prove every case.

Simulation drives selected inputs and checks outputs. For the counter, test reset priority, hold, repeated increments, and 15-to-0 wraparound. A self-checking testbench compares the design against the reference

rule instead of only inspecting a plausible waveform.

Formal property checking asks whether stated properties hold for behaviors admitted by a model and assumptions; the result may be bounded or unbounded depending on the method. A proof with inappropriate clock, reset, or environmental assumptions can miss the real problem.

Equivalence checking compares representations, often RTL against a synthesized or modified netlist. It detects transformations that change behavior under the comparison assumptions. It does not establish that the starting specification was correct; reset/initial-state and test-mode assumptions matter.

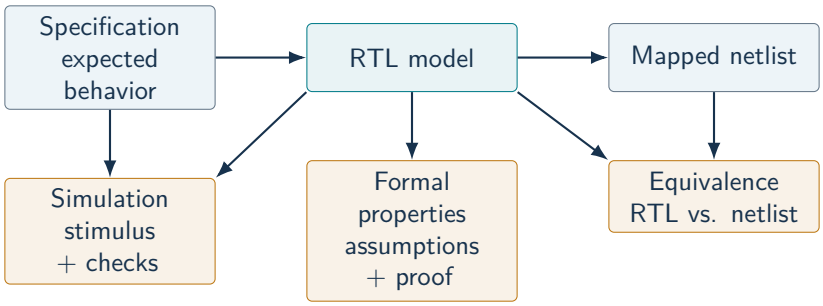


Figure 7: Complementary verification questions; none alone replaces timing or physical verification.

Coverage guides missing cases, transitions, or properties. A percentage is meaningful only relative to the coverage model. Lint checks coding issues. Clock-domain crossing (CDC) and reset-domain crossing checks examine structural hazards. Gate-level simulation can help with particular reset, test, or timing-annotation questions, but does not replace STA.

For this counter. If `en` comes from an unrelated clock or external source, resolve that crossing at the interface. A suitable two-flop synchronizer reduces metastability propagation for a single-bit level, but can miss narrow pulses and does not make an arbitrary multibit transfer coherent. Handshakes, pulse-transfer schemes, or asynchronous FIFOs may be needed [11]. Synchronization is an architectural choice, not a timing-exception shortcut.

App practice: Turn a plausible trace into explicit checks

1. Select **4-bit event counter** in **RTL & behavior (02)** and restart. Write a checkpoint table with stimulus, predicted count, and observed count.
2. Check increment twice, a disabled edge, and reset with enable high. Record the expected sequence 1, 2, 2, 0.
3. Restart, leave Enable on and Reset off, and step 15 times to reach 15. Step once more and check 0: this is the wraparound case.
4. Switch between **Word level** and **Gate level** at the same state. Step once in either view and compare the resulting outputs in both views.
5. Export a CSV for your chosen stimulus sequence. Name a behavior or assumption that these finite checks do not establish, such as CDC safety or all possible initial conditions.

Expected observation. Both compiled abstractions agree for the exercised sequence. **Explain.** Agreement for selected stimuli is simulation evidence, not a formal equivalence proof, coverage report, or timing verdict.

5 Synthesis chooses an implementation from a library

Lecture window: 10–12:30 minutes. *Turn operators into cells; explain PPA tradeoffs.*

Plain-language starting point. Synthesis is the hardware compiler. It chooses a network that implements the rule. A standard-cell library supplies the physical building blocks that a chip implementation can use.

Synthesis parses/elaborates RTL, resolves hierarchy and widths, infers combinational/sequential structures, optimizes logic, and maps into available cells. A mapped counter could contain four flip-flops, increment logic, and selection logic. Its exact cell count, carry structure, and use of integrated-enable/reset cells depend on the library and settings. ASIC Explorer supplies an actual *generic* Yosys mapping to inspect; it does not map this counter into a foundry-characterized standard-cell library.

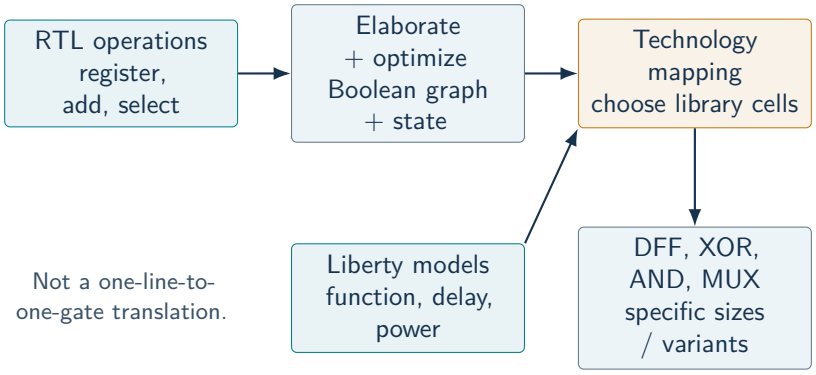


Figure 8: Optimization and library-based technology mapping.

Yosys documentation distinguishes abstract RTL cells from concrete target-library cells and explains Liberty-based mapping [3]. A register, addition, or conditional is not a guaranteed single physical cell.

Synthesis optimizes against timing, area, and electrical constraints. **Power, performance, and area (PPA)** interact. A larger driver can reduce one delay but increase area and input capacitance. Buffering can reduce a long net's delay but costs resources. Pipelining can shorten combinational paths but changes latency and adds state; it must satisfy the specification.

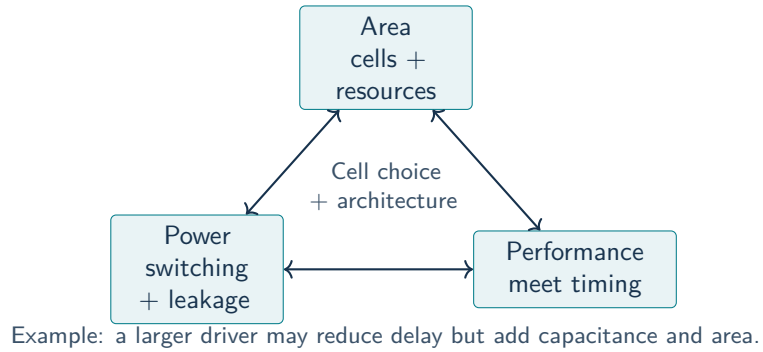


Figure 9: PPA tradeoffs; the constraints determine what is acceptable.

Read the outputs. Inspect inferred state, mapped netlist, area summary, warnings, timing, and unconstrained paths. A clean exit is not a design verdict. Synthesis interconnect estimates become more physical after placement and routing.

App practice: Inspect and change the synthesized hardware

1. Open **Gates & mapping (03)**, select **4-bit event counter**, restart, and choose **Word level**. Select the adder and inspect its highlighted RTL association.
2. Choose **Gate level**. Locate the flip-flops and combinational gates. Open **Synthesis report**; record the word-level and generic mapped cell counts. Diagram inputs, outputs, and display helpers are not all counted as synthesized cells.
3. Select **Counter · increment by 2**, restart, and step twice. Observe 2, then 4, with the least significant bit staying 0; compare the mapped structure and report.
4. Return to the ordinary counter and change `count + 4'd1` to `count + 4'd3` in the editor. Keep Top module as `event_counter`. Before compiling, notice that live controls are disabled and the circuit is labeled as the last compiled source.
5. Click **Synthesize**, wait for success, and step once: count becomes 3. Inspect the new constant and circuit. Use **Restore example source** to return to the prepared counter.

Explain. The circuit is computed from HDL, not selected from a picture. These are generic logic operators/-cells; their counts do not establish physical area, foundry timing, or PPA improvement.

6 Technology data connects logical and physical tools

Lecture window: Supporting reference outside the core. *Teach file purposes, not just extensions.*

Plain-language starting point. Different tools need different descriptions of the same components. One file describes behavior, another timing, and another dimensions. Read the purpose before memorizing the extension.

A **process design kit (PDK)** describes the manufacturing technology through device models, layer definitions, layout rules, and verification/extraction data. A compatible **standard-cell library** supplies gates, registers, and supporting cells with logical, timing, power, and physical views. These are related but distinct concepts.

Artifact	What it tells a tool
RTL	Intended behavior and state; not mask geometry.
Mapped netlist	Named library cells and pin-to-net connectivity.
Liberty (`.lib`)	Cell function, timing arcs, setup/hold, power, electrical models, units, and operating conditions.
Tech / cell LEF	Routing layers/rules and abstract cell sizes, pins, obstructions.
DEF / database	Design floorplan, positions, orientation, and routing.
SDC	Clocks, interface timing, margins, justified exceptions.
SPEF	Extracted wire RC network; not cell-delay models.
GDSII / OASIS	Detailed layout geometry/hierarchy for handoff.
Rule decks / models	DRC, LVS, extraction, device simulation, other process checks.

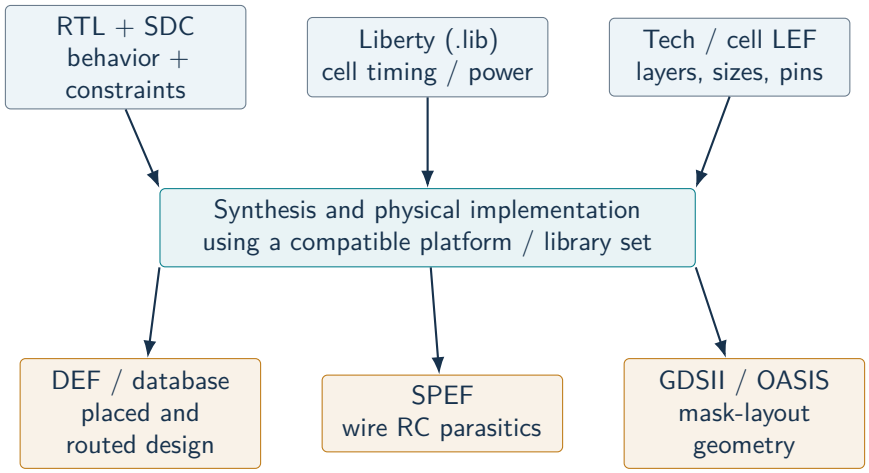


Figure 10: Important artifact roles; a conceptual map rather than an executable recipe.

Check characterized **process, voltage, temperature (PVT)**, interconnect corners, and required functional/test modes. "Slow for setup, fast for hold" is a starting intuition; the actual models determine worst cases.

Educational platforms and foundry acceptance

Foundry acceptance needs the agreed process, validated decks, and signoff evidence. Public SKY130 documentation distinguishes open-release status from the underlying process's manufacturing history [10].

App practice: Classify what the app actually produces

1. In **Gates & mapping (03)**, restore the counter, click **Synthesize**, and open **Synthesis report**. Locate the generic mapping command and reported cells.
2. Save the source with **Download HDL source** and the current schematic with **Download circuit SVG**. Classify the first as RTL and the second as a visualization of compiled connectivity.
3. Open **Routing & RC (07)**. Compare this drawing with the SVG and the artifact table above. State which file would carry real extracted wire RC and which would carry detailed layout.
4. Write down the missing inputs for a foundry implementation: compatible Liberty/LEF and PDK views, constraints, physical databases, extraction models, and approved verification decks.

Explain. An SVG is neither GDSII nor a netlist interchange file. The app supplies no Liberty-characterized mapping, LEF/DEF layout, SPEF extraction, or foundry signoff report. Its generic synthesis report still gives useful implementation evidence at its stated abstraction.

7 Timing: two registers and the path between them

Lecture window: 13:30–15 minutes. *Build the path model before presenting equations.*

Plain-language starting point. A value leaves one register, travels through logic and wires, and reaches another. Ask two questions: is the new value ready before the next sample, and does the old value survive briefly after this sample?

The launch flip-flop changes its output after an active edge. The change propagates through combinational

cells and wires to a capture flip-flop. New data must arrive early enough for setup; the old value must remain stable long enough for hold.

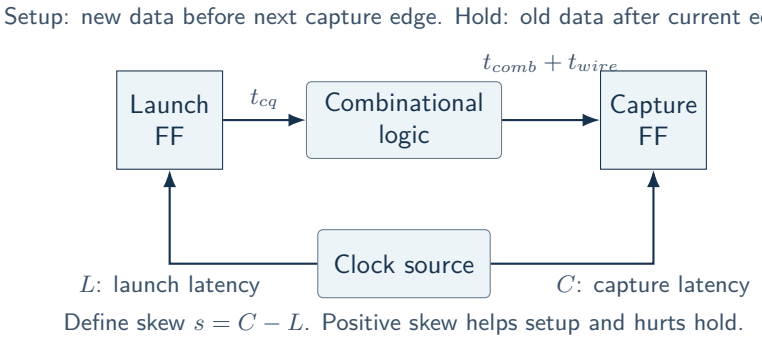


Figure 11: One path with explicit launch and capture clock latencies.

Let T be period, L launch clock latency, C capture clock latency, and $s = C - L$ path-specific skew. Let t_{cq} be clock-to-Q, and $t_{data} = t_{comb} + t_{wire}$ the remaining data delay. Cell delay depends on slew/load; wire delay depends on RC. Max and min represent late and early path delays and are not interchangeable.

Setup limits the latest arrival before the intended capture edge. **Hold** limits the earliest replacement of the old value after the current capture edge. **Static timing analysis (STA)** analyzes constrained paths using timing models without enumerating functional input sequences. It needs correct clock relationships; it is not a functional proof. OpenSTA documents netlist, Liberty, SDC, and SPEF interfaces for this analysis [4].

The following equations assume a single-cycle, same-frequency, rising-edge register-to-register path. Separate nonnegative margins are U_{setup} and U_{hold}. Real tools also handle generated clocks, different edges, variation, common-path effects, conditional arcs, and exceptions.

Teacher wording. "Setup says the next value must not be late. Hold says the old value must not disappear too early. Both must pass."

App practice: Read arrival and required time before slack

Controls for this activity. If a step changes a slider, click **Show equations and sliders** in Guided view first.

1. Open **Timing (04)** and click **Before routing**. Identify launch FF, logic, wire, optional buffer, and capture FF.
2. Read the setup bar: arrival is 1.25 ns and required time is 1.55 ns. Subtract them and predict setup slack before reading its card.
3. Find both Setup slack and Hold slack cards. Explain why the same baseline can pass setup (+0.30 ns) and fail hold (−0.04 ns).

Explain. Setup and hold compare different arrivals against different requirements. The app evaluates the illustrated path equations; it does not run STA on the editable HDL or the six-cell placement model.

8 Setup: calculate the latest allowed arrival

Lecture window: 15–16:30 minutes. *Work one numerical example with the students.*

Plain-language starting point. Setup is a deadline for the next update. In our example, arrival at 1.25 ns beats the 1.55 ns deadline by 0.30 ns. Arrival at 1.60 ns misses that same deadline by 0.05 ns.

The calculation below explains where the deadline comes from.

$$\begin{aligned} Arrival_{setup} &= L + t_{cq,max} + t_{data,max}, \\ Required_{setup} &= T + C - t_{setup} - U_{setup}, \\ Slack_{setup} &= Required_{setup} - Arrival_{setup} \\ &= T + s - t_{cq,max} - t_{data,max} - t_{setup} - U_{setup}. \end{aligned}$$

Nonnegative slack meets the modeled requirement. Negative slack means the latest modeled data arrives too late.

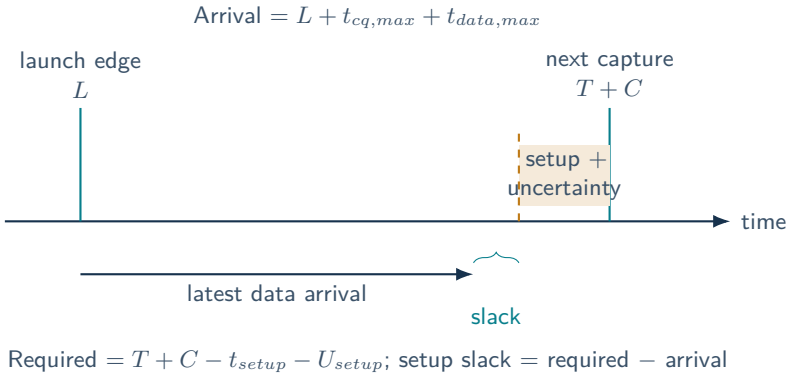


Figure 12: Setup uses the next capture edge. The sketch is not scaled to the numerical example.

Term	Pre-route	Post-route	Interpretation
T	2.00 ns	2.00 ns	Illustrative target period
$t_{cq,max}$	0.15 ns	0.15 ns	Launch clock-to-Q
$t_{comb,max}$	0.85 ns	0.85 ns	Cell-path delay
$t_{wire,max}$	0.25 ns	0.60 ns	Before / after routing model
t_{setup}	0.15 ns	0.15 ns	Capture setup requirement
U_{setup}	0.30 ns	0.30 ns	Setup margin
s	0.00 ns	0.00 ns	Zero skew for this exercise

Before routing, consumed budget is $0.15+0.85+0.25+0.15+0.30 = 1.70$ ns. Arrival is 1.25 ns and required time is 1.55 ns, giving $\boxed{+0.30 \text{ ns}}$ slack. After routing, arrival is 1.60 ns; slack is $1.55 - 1.60 = \boxed{-0.05 \text{ ns}}$. These inputs match ASIC Explorer's **Before routing** and **After routing** presets.

Prediction question 2

"The RTL has not changed. Why did timing fail after routing?" The estimated wire delay was too small; the physical route adds RC delay. Let students calculate the new slack before revealing −0.05 ns.

The table isolates wire delay for teaching. Real routing can also change slew, cell delay, clock skew, and buffering. These values represent a **generic path**, not a measured path in the counter or a particular process.

Repairs. Shorten a net by improving placement; resize or buffer a limiting segment; restructure logic; or revise architecture if the specification permits. Relaxing period requires system-level permission. An unjustified false path hides a failure rather than repairing it.

App practice: Reproduce the setup calculation

Controls for this activity. If a step changes a slider, click **Show equations and sliders** in Guided view first.

1. In **Timing (04)**, choose **Before routing**. Record period 2.00 ns, Wire delay (max) 0.25 ns, skew 0, and Added hold-buffer delay 0.
2. Use the table above to calculate arrival 1.25 ns, required time 1.55 ns, and slack +0.30 ns. Match all three with the bar/card.
3. Choose **After routing**. Only the modeled maximum wire delay changes to 0.60 ns. Predict arrival 1.60 ns and slack −0.05 ns before checking the display.
4. Keep that case and raise **Clock period** to 2.10 ns. Setup becomes +0.05 ns while hold remains −0.04 ns. Explain what changed in the contract.

Explain. More wire delay consumes margin; a longer period gives setup more time. Changing the target to 2.10 ns is a hypothetical contract change, not evidence that the design meets its original 2.00 ns requirement. The maximum-wire slider does not change minimum delay.

9 Hold: protect the old value at the current edge

Lecture window: 16:30–18 minutes. *Keep the same skew convention.*

Plain-language starting point. Hold protects the value just sampled at the current edge. Data that changes too soon can violate that protection even if it easily meets the next deadline. Slowing the next clock edge does not fix this short-path problem.

$$\begin{aligned} Arrival_{hold} &= L + t_{cq,min} + t_{data,min}, \\ Required_{hold} &= C + t_{hold} + U_{hold}, \\ Slack_{hold} &= Arrival_{hold} - Required_{hold} \\ &= t_{cq,min} + t_{data,min} - s - t_{hold} - U_{hold}. \end{aligned}$$

The slack sign convention differs from setup because new data must arrive *after* the hold requirement. Later capture clock arrival gives more setup time but extends how long the old data must remain stable.

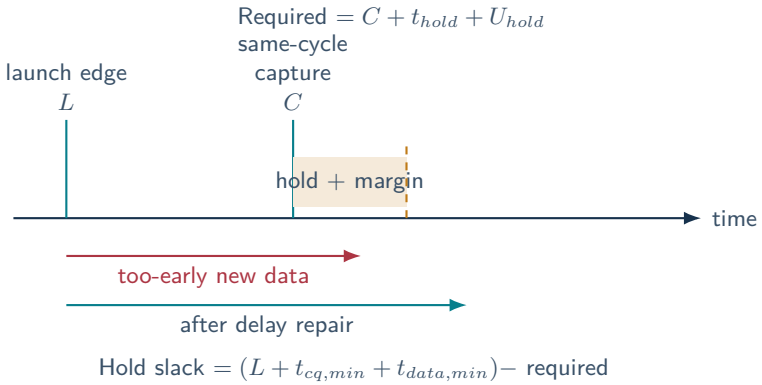


Figure 13: Hold uses the current edge relationship. New data arriving before the end of the window is too early.

Use the app's baseline: $t_{cq,min} = 0.06$ ns, $t_{comb,min} = 0.03$ ns, $t_{wire,min} = 0.01$ ns, $s = 0$, $t_{hold} = 0.12$ ns,

and $U_{hold} = 0.02$ ns:

$$Slack_{hold} = 0.06 + 0.03 + 0.01 - 0 - 0.12 - 0.02 = \boxed{-0.04 \text{ ns}}.$$

The **Repair hold** preset adds 0.06 ns of modeled buffer delay and gives hold slack +0.02 ns. It begins from the pre-route baseline. The app adds that delay to both minimum and maximum paths, so setup slack falls from +0.30 to +0.24 ns. Actual buffer delay depends on corner, slew, and load; recheck every affected scenario.

Slowing the clock is not a general hold repair

There is no period T in this same-edge hold equation. Lower frequency can help setup but ordinarily does not remove this failure. Use an appropriate data-delay or clock/data-path repair and verify setup again.

App practice: Try a repair that frequency cannot provide

Controls for this activity. If a step changes a slider, click **Show equations and sliders** in Guided view first.

1. In **Timing (04)**, click **Before routing**. Calculate earliest arrival $0.06+0.03+0.01 = 0.10$ ns and required hold time $0.12 + 0.02 = 0.14$ ns.
2. Predict and check hold slack -0.04 ns. Click **Slower clock**: setup becomes +1.30 ns, but hold remains -0.04 ns.
3. Click **Repair hold**. This loads an independent pre-route case with 0.06 ns added buffer delay; hold becomes +0.02 ns and setup becomes +0.24 ns.
4. Click **After routing**, then move **Added hold-buffer delay** to 0.06 ns manually. Now hold is +0.02 ns but setup is -0.11 ns. Compare with the previous preset.

Explain. Delay can repair the short path while worsening the long path. Preset buttons reset the case; the slider lets you apply a repair to the current case. Real buffers require min/max characterization across corners rather than this equal-delay simplification.

10 Constraints express the timing contract

Lecture window: 20–21 minutes. *Show interface budgets and a short SDC example.*

Plain-language starting point. The designer must tell the tools how much time the circuit and its interfaces have. Constraints are the timing part of the contract. They express requirements rather than making the circuit faster.

Clocks, relationships, min/max interface budgets, uncertainty, input transitions, and output loads tell the tool what operation is required. Good internal timing alone does not guarantee a working chip-to-chip interface.

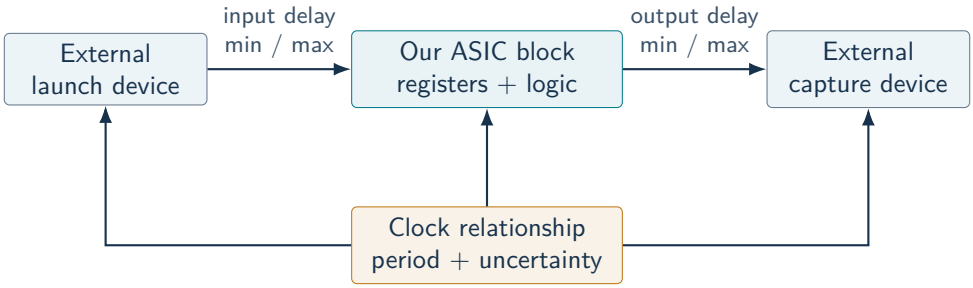


Figure 14: Timing and electrical assumptions at a block interface.

Listing 2: Illustrative SDC; assumes the loaded library's time unit is ns.

```
create_clock -name core_clk -period 2.00 [get_ports clk]
set_clock_uncertainty -setup 0.30 [get_clocks core_clk]
set_clock_uncertainty -hold  0.02 [get_clocks core_clk]
set_input_delay  -clock core_clk -max 0.30 [get_ports {rst en}]
set_input_delay  -clock core_clk -min 0.05 [get_ports {rst en}]
set_output_delay -clock core_clk -max 0.40 [get_ports {count[*]}]
set_output_delay -clock core_clk -min 0.00 [get_ports {count[*]}]
set_input_transition 0.05 [get_ports {rst en}]
# Add set_load from the actual receiver/board model,
# using the loaded library's capacitance unit.
```

The zero output min value represents an illustrative external hold requirement of zero. Real values depend on receiver/board delays and can be negative. Verify that port patterns match all four output bits and units match the library. This is a teaching contract, not a complete production SDC.

Synchronous reset is a timed input here. An asynchronous reset instead needs recovery/removal and safe-release considerations. A **false path** excludes a genuinely untimed relationship. A **multicycle path** changes intended launch/capture edge pairing; derive setup and hold adjustments together.

Report check. Inspect missing clocks, unconstrained endpoints, electrical limits, and setup/hold paths in every required scenario. Worst negative slack (WNS) and total negative slack (TNS) summarize failures, but do not prove constraint coverage or correctness. Required constrained slack should be nonnegative and TNS zero, subject to actual signoff criteria.

App practice: Change a requirement rather than the logic

Controls for this activity. If a step changes a slider, click **Show equations and sliders** in Guided view first.

1. Open **Timing (04)**, click **Before routing**, and reduce **Clock period** from 2.00 to 1.50 ns. Predict setup −0.20 ns and unchanged hold −0.04 ns.
2. Reload **Before routing**. Set **Capture clock skew** to +0.10 ns: setup becomes +0.40 ns and hold −0.14 ns.
3. Compare the period and margins with the SDC listing. Explain the roles of `create_clock`, setup/hold uncertainty, and input/output budgets.
4. List the constraints the app's sliders cannot express: interface min/max delays, loads/slew, generated clocks, justified exceptions, and multiple scenarios. Keep the SDC exercise separate from the scalar timing experiment.

Explain. The app does not parse SDC or diagnose unconstrained paths. Its setup/hold uncertainty defaults match this listing, but its generic internal path does not model the listed external interface budgets. A better-looking card after changing a requirement is not a repair under the old requirement.

11 Design for test: access to manufactured internal state

Lecture window: 27:30–29 minutes. *Separate manufacturing faults from design bugs.*

Plain-language starting point. A correct design can still be manufactured with a defect. Test access lets equipment set and inspect internal state to look for those defects. Scan is one way to provide that access.

Functional verification checks the design model. Manufacturing test checks whether a manufactured instance contains detectable defects. **Design for test (DFT)** makes internal state controllable and observable. Plan it early: it changes area, ports, wiring, timing, and possibly clocks.

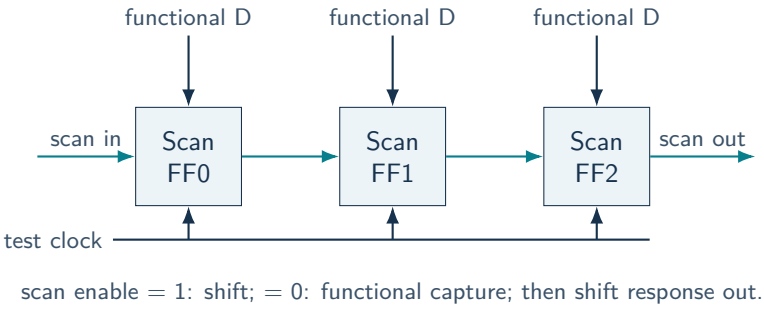


Figure 15: Simplified scan chain. Each flop selects functional or scan data under scan-enable control.

In **shift mode**, a test clock loads a known pattern. In **capture mode**, the logic generates a response captured into the scan registers. Shift that response out and compare it with the expected response. Automatic test-pattern generation (ATPG) targets a selected fault model, such as stuck-at or transition faults. Fault coverage describes that modeled population, not every possible physical defect.

Scan insertion/replacement can occur around synthesis and before placement; physical positions can guide stitching/reordering. OpenROAD documents scan-flop replacement before placement and location-aware chain construction after placement [7]. Other flows arrange these tasks differently. Memories can need memory built-in self-test (MBIST); boundary scan has a different interface-testing role.

The counter has four state bits. The three-flop drawing shows the mechanism clearly rather than claiming its final mapped chain. Scan does not prove the RTL specification is right; simulation does not prove each fabricated instance is defect-free.

Quick check

"If simulation says the design is correct, why test each manufactured chip?" Fabrication can introduce defects. The two activities answer different questions.

App practice: Use a mux to understand selection, then reason about scan

1. In **RTL & behavior (02)**, select **2-to-1 multiplexer** and restart. Set input `a=1`, `b=0`, and `sel=0`; verify `y=1`.
2. Change `sel` to 1; verify `y=0`. Relate this selection to choosing functional data or scan data at a scan-flop input.
3. Use Figure 15 to describe shift, capture, and shift-out. Identify what would have to be added to the counter for controllability and observability of internal state.

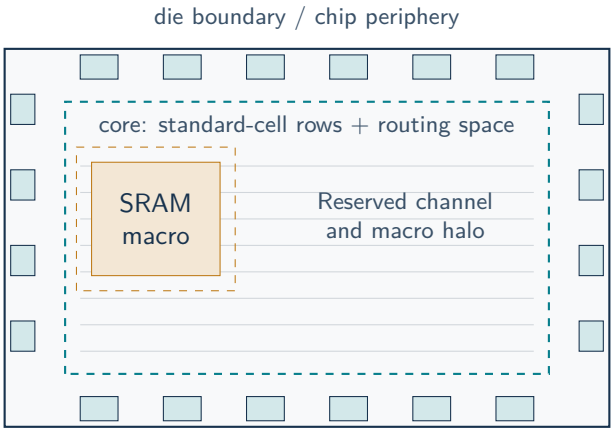
Explain. The mux is an analogy for the input selection mechanism. The app has no scan insertion, chain shifting, ATPG, or fault-coverage feature. The diagram-and-reasoning step is necessary; toggling a mux does not implement DFT or test a manufactured device.

12 Floorplanning: reserve a feasible physical home

Lecture window: 21–22:30 minutes. *Provide room for more than logic cells.*

Plain-language starting point. Floorplanning is reserving a sensible home for the design. The chip needs space for components, major blocks, power delivery, and the connections that will be added.

Floorplanning defines die/core boundaries, I/O or block pins, macro positions, cell rows, channels, and placement restrictions. **Power planning** provides required rails, straps, rings, and connections. Details differ for a block and a complete chip.



Generic larger-chip example; the teaching counter has no SRAM macro.

Figure 16: A generic larger-chip floorplan with macro halo, rows, channels, and peripheral I/O; not the counter's measured layout.

Utilization relates standard-cell area to the appropriate available placement area. For a macro-free toy

core, a first estimate is:

$$A_{core} \approx \frac{A_{cells}}{u}.$$

For $A_{cells} = 12{,}000\,\mu\text{m}^2$ and target $u = 0.60$, $A_{core} \approx 20{,}000\,\mu\text{m}^2$. A square side is about $141.4\,\mu$m before site/row alignment. This generic arithmetic is not the counter's area. Macros, halos, blockages, power structures, and later cells alter usable area and final sizing.

High utilization can save footprint but leave insufficient routing/repair resources. Excessive empty space can lengthen wires and increase area. A feasible target depends on connectivity, macros, the routing stack, clock/power requirements, and platform.

Physical-only cells. Tap/well-tie and endcap cells meet process/row requirements; tie cells supply safe constants; filler/decap cells support continuity and supply behavior as required. Insertion stages vary. Reserve room for clock buffers, timing repair, and test logic rather than sizing solely from initial synthesis area.

Transition. "We reserved the neighborhoods. Placement now assigns legal addresses to the cells."

App practice: Inspect space before optimizing positions

1. Open **Floorplan & place (05)** and click **Reset layout**. Find die boundary, core, rows, the illustrated power rails, and the six labeled teaching cells.
2. Set **Illustrative utilization** to 60%, then 80%. Observe the change in gray filler and the explanatory text.
3. Check the connection-length readout while leaving the six named cells unmoved: it stays 1020 units. The filler illustration did not change those connections.
4. Explain where clock buffers, signal routes, power structures, and later repairs need space. Return utilization to 60% for the next experiment.

Explain. This slider illustrates occupied space; it does not calculate a real core area, pack library cells, or measure congestion. Use the area equation above separately. The macro/halo in Figure 16 is supporting conceptual detail and is not an editable app feature.

13 Placement: locations alter electrical behavior

Lecture window: 22:30–23:30 minutes. *Explain legality, wirelength, and congestion.*

Plain-language starting point. Placement gives each cell an address. Moving connected cells can change wire distance while preserving what connects to what. A shorter drawing is a clue about wiring cost, not a complete timing result.

Global placement selects approximate locations using wirelength, density, timing, and congestion objectives. Overlap can remain during optimization. **Legalization/detailed placement** aligns cells to allowed sites/rows, resolves overlaps, obeys orientations/blockages, and refines positions.

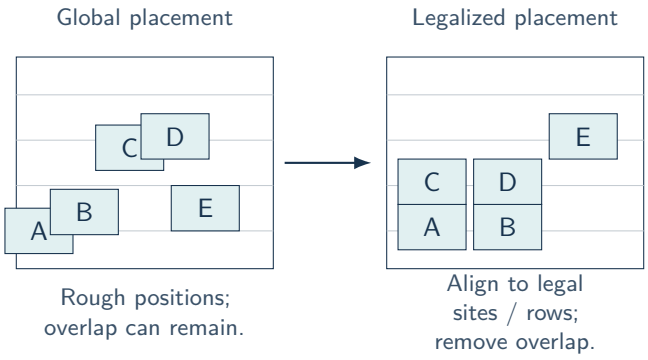


Figure 17: Approximate versus legalized placement; cell names and sizes are schematic.

Nearby connected cells often mean shorter wires, but packing critical logic tightly can congest routing or stress power delivery. Minimum distance alone is not the complete objective.

For the counter, keep register/increment/select logic sensibly related, permit clock/pin access, and leave wiring/repair space. Output-pin positions affect interface timing too. A location includes orientation and pin-access consequences, not only (x, y).

Inspect density/congestion maps, estimated timing, long/high-fanout nets, slew/capacitance violations, and illegal/unplaced cells. Severe congestion can require revisiting the floorplan instead of repeatedly routing an infeasible placement.

App practice: Measure the effect of moving a cell

1. In **Floorplan & place (05)**, click **Reset layout**. Record Manhattan connection length 1020 units.
2. Focus **DFF 0** and press the right arrow once. It moves 5 teaching units and total connection length becomes 1015; the arrow moves the cell rather than navigating lessons. You can also drag a cell.
3. Click **Reset layout**, then **Compact placement**. Record 455 units and compare it with 1020. The metric sums the five model connections' Manhattan distances.
4. Explain why shorter connections can be helpful yet cannot, by themselves, establish a faster, legal, or routable chip.

Explain. The readout measures actual coordinate changes within this teaching model. It is not an extracted route length, timing calculation, legalization result, congestion score, or comparison of the synthesized counter's implementations.

14 Clock tree synthesis: deliver edges to the registers

Lecture window: 23:30–24:30 minutes. *Connect the clock network to timing equations.*

Plain-language starting point. The clock must travel to the registers too. A clock tree distributes it through wires and buffers. Controlled arrival differences matter because registers use those edges to sample data.

RTL often models all registers seeing an ideal simultaneous clock. A real clock must drive distributed capacitive loads through physical wires. **Clock tree synthesis (CTS)** organizes the network and inserts suitable clock buffers/inverters to manage latency, skew, transition time, fanout, routing, and power. OpenROAD's CTS documentation provides one concrete implementation [5].

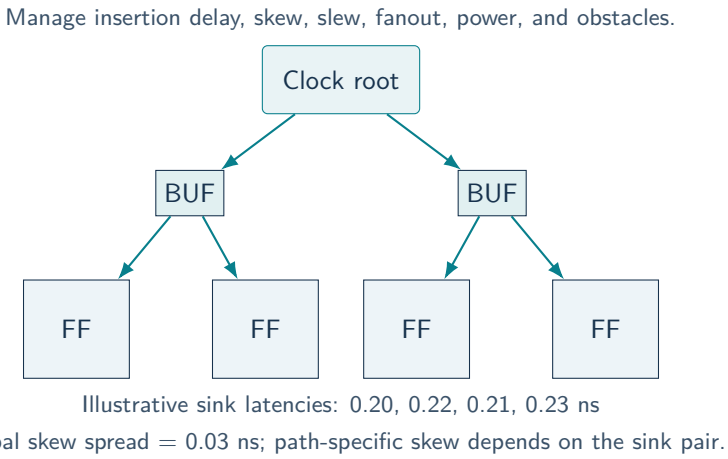


Figure 18: Conceptual buffered clock distribution; real trees depend on sinks, loads, obstacles, and constraints.

Latency/insertion delay is time from a clock reference to a sink. **Skew** is the difference between two sink arrivals. **Jitter** is temporal edge variation, not spatial skew. Uncertainty must be budgeted consistently with ideal versus propagated clock modeling.

If launch latency is 0.20 ns and capture latency is 0.23 ns, $s = +0.03$ ns: the simple setup margin increases by 0.03 ns, while hold margin decreases by 0.03 ns. Later capture is therefore not always beneficial.

After CTS, use appropriate propagated-clock analysis and repair exposed violations. Avoid double-counting physical skew in both propagated arrivals and an unchanged pre-CTS allowance. Clock/signal routing still affects final extracted delays.

Depth. Clocks toggle often and drive substantial capacitance. Clock gating uses approved integrated clock-gating cells and proper checks; a casual combinational AND can glitch. The counter's RTL data enable does not guarantee a particular clock-gating implementation.

App practice: Distinguish sink spread from path-specific skew

1. Open **Clock tree (06)** and click **Reset layout**. Read sink arrivals 0.08, 0.18, 0.28, and 0.38 ns. The maximum minus minimum is 0.30 ns.
2. Click **Build balanced clock tree**. All example sink arrivals become 0.20 ns and the displayed spread is 0.00 ns. Identify the drawn clock buffers.
3. Open **Timing (04)**, click **Before routing**, and set Capture clock skew to +0.10 ns. This illustrates a path with capture 0.10 ns later than launch, as for two adjacent sinks in the earlier example.
4. Verify setup +0.40 ns and hold −0.14 ns. Restore skew to 0 and explain why a later capture edge has opposite consequences.

Explain. Total sink spread and a chosen launch/capture skew are different quantities. The clock-tree drawing assigns illustrative arrivals; it does not run CTS or account for real load, slew, obstacles, or RC. The timing slider is independent of the clock drawing.

15 Routing: build wires and vias that obey the technology

Lecture window: 24:30–26 minutes. *A logical net becomes a physical connection.*

Plain-language starting point. Routing turns the wiring list into real metal paths. Different layers

provide room to cross and connect. Vias conduct between layers.

A net states which pins connect. Routing chooses paths, layers, widths, and vias. **Global routing** allocates approximate paths/resources and estimates congestion. **Detailed routing** realizes track-level geometry, pin access, and vias while observing design rules. Routing repair does not replace final complete physical checks.

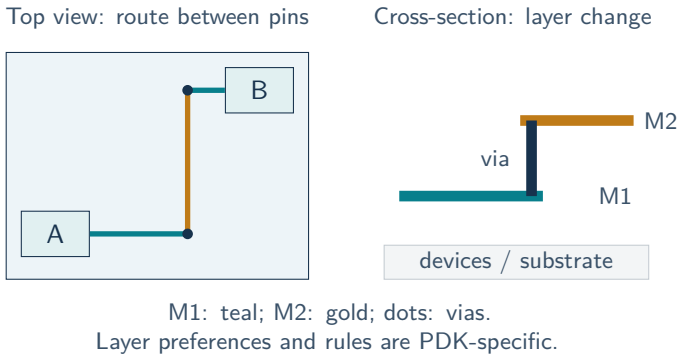


Figure 19: Top-view route and conceptual layer cross-section. A via connects layers where specified.

Layers can have different preferred directions, thicknesses, resistances, capacities, and spacing rules. Higher layers may be suitable for long/global nets, but no universal rule makes every higher-layer route faster or every layer pair alternate directions. Technology and routing strategy determine the choices.

Constraints include width/spacing, via enclosure, short/open avoidance, antenna rules, and available resources. A detour can satisfy connectivity while raising delay/capacitance. **Crosstalk** couples neighboring-wire switching into delay or noise. Spacing, shielding, layer changes, or buffering can help but have resource/electrical costs.

Antenna depth. Some fabrication sequences accumulate charge on conductor shapes connected to a gate, threatening oxide integrity. Layer jumps or antenna diodes are technology-dependent repairs. This process issue differs from operating-time crosstalk.

App practice: Separate logical connection from layer geometry

1. Open **Routing & RC (07)** and click **Reset layout**. Leave both **M2 · horizontal wires** and **M3 · vertical wires** checked.
2. Trace one connection from a cell through a horizontal segment, a square via marker, and a vertical segment. Explain what the via connects.
3. Uncheck M2, then recheck it and uncheck M3. Identify what each visibility control hides. Restore both before continuing.
4. Compare this view with a gate schematic in **Gates & mapping (03)**. State which representation describes connectivity and which illustrates an imagined physical realization.

Explain. Visibility toggles do not physically remove a route or run open/short checks. The layer directions and square via drawings are examples, not PDK rules. The app does not perform global/detailed routing or antenna checks.

16 Extraction: replace wire guesses with physical models

Lecture window: 24:30–26 minutes. *Return to the setup failure caused by wire delay.*

Plain-language starting point. A real wire has resistance and capacitance, so a signal takes time to change voltage along it. Extraction estimates those electrical effects from the routed geometry, then timing analysis uses the estimates.

Parasitic extraction derives interconnect resistance/capacitance from routed geometry and process models. Cell models remain in Liberty; extracted RC describes the wires. SPEF is a common exchange format. OpenROAD documents RC extraction and SPEF output [6].

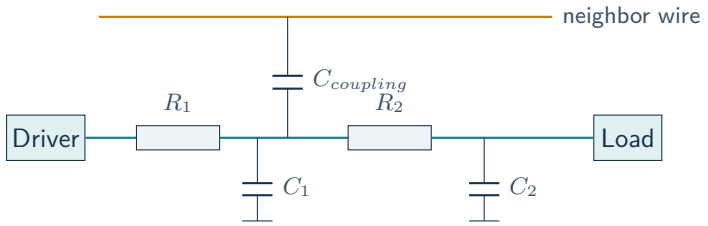


Figure 20: Simplified distributed RC with ground and coupling capacitance. Real networks can contain many segments.

For an isolated lumped model, a rough time constant is RC. For $R = 100\,\Omega$ and $C = 20$ fF, $RC = 2$ ps. This dimensional illustration is not a signoff delay formula: topology, driver/load behavior, thresholds, coupling, and variation determine actual delay.

Repeat STA using extracted parasitics, required corners, and clock models. Our setup exercise increased modeled wire delay from 0.25 to 0.60 ns, consuming margin and producing −0.05 ns slack. The app illustrates this effect by changing a scalar; it does not extract these values from its routing drawing. A passing pre-route estimate is not the final timing verdict.

Depth. Manufacturing-density metal fill can affect capacitance. Use parasitics consistent with the final approved geometry and required fill-aware extraction method. A stale SPEF from a previous route cannot establish final timing.

App practice: Connect an RC model to a timing consequence

1. In **Routing & RC (07)**, reset the layout, show both metal layers, and click **Reveal parasitic RC**. Locate the R and C labels.
2. Explain what information a real extractor needs to assign resistance and capacitance. The overlay supplies symbols, not measured component values or a SPEF file.
3. Open **Timing (04)** and choose **Before routing**, then **After routing**. Compare maximum wire delay 0.25/0.60 ns and setup slack +0.30/ − 0.05 ns.
4. Write one sentence linking physical geometry, extracted RC, and a timing model. Identify the step the app illustrates rather than executes.

Explain. The scalar timing change represents a possible effect of extraction. It is not calculated from the RC overlay or from cell movement. A real comparison requires matched geometry, models, extraction, and timing reports.

17 Power and reliability constrain implementation

Lecture window: 27:30–29 minutes. *Power is also a delivery problem.*

Plain-language starting point. Cells need a reliable power supply as well as data and clock signals. Switching uses energy; resistance in the supply network can reduce the voltage reaching the cells. Power is an implementation requirement.

A switched-capacitance approximation is:

$$P_{switch} \approx \alpha C_{eff} V^2 f.$$

Here α counts average charging (0-to-1) events per cycle under this convention, C_{eff} is effective switched capacitance, V is supply, and f is frequency. If activity counts all transitions, the prefactor convention differs. Total power also includes internal-cell/short-circuit effects as modeled and leakage. Workload assumptions matter.

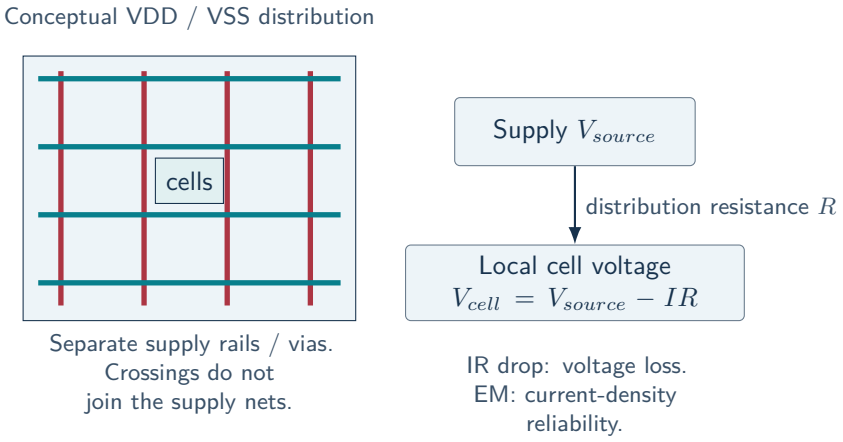


Figure 21: Conceptual supply distribution and local voltage loss. Each supply net has separate detailed connectivity.

IR drop reduces voltage reaching cells. Lumped values $I = 20$ mA and $R = 2\,\Omega$ give 40 mV drop. Actual static/dynamic grid analysis needs a network, supply sources, and activity models. Local droop can affect timing and noise margin.

Electromigration (EM) concerns material transport under current flow. Limits depend on geometry, temperature, current density, duty cycle, and process. Timing-clean wiring can still violate supply/reliability requirements. Improve straps/vias/supply connections or redistribute activity according to actual analysis.

Takeaway. Power is both an energy budget and a physical delivery/reliability constraint. An estimate using assumed switching is not a measured workload result.

App practice: Observe switching without calling it a power measurement

1. In **RTL & behavior (02)**, select **4-bit event counter**, restart, and step twice. Observe changes in count and clock.
2. Turn **Enable** off and step twice more. Count is held while the clock still toggles. Explain why a data enable does not automatically demonstrate clock gating.
3. Use Figure 21 and $P_{switch} \approx \alpha C_{eff} V^2 f$ to list the missing quantities needed to estimate power and analyze voltage droop or current density.

Explain. Functional values can motivate activity reasoning, but this short trace is not a representative workload or a power report. The app has no capacitance/voltage characterization, IR-drop analysis, EM analysis, or clock-gating implementation measurement.

18 Signoff: several independent questions

Lecture window: 29–31 minutes. *Distinguish functional checks, STA, DRC, and LVS.*

Plain-language starting point. Each check asks its own question. Behavior checks the rule; timing checks when values arrive; DRC checks manufacturing geometry; LVS checks devices and connections against the reference circuit.

Signoff completes and reviews checks required for the intended implementation/handoff. Tool, deck, and acceptance requirements come from the platform and foundry agreement.

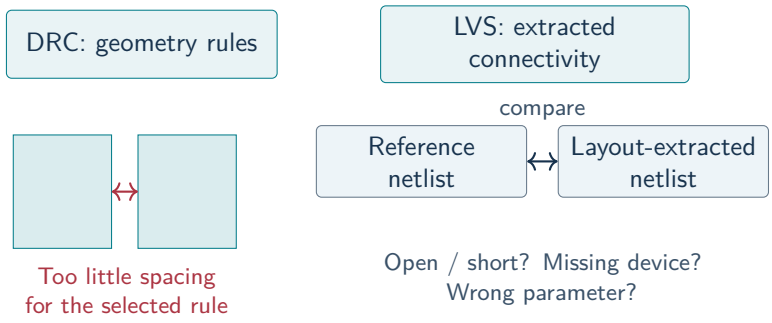


Figure 22: DRC checks geometry; LVS compares extracted circuit structure with a reference. KLayout documents both [8, 9].

Check	Question	Typical issue
Functional	Does the model meet behavior requirements?	Wrong reset/overflow rule
Equivalence	Was reference behavior preserved?	Modified logic; wrong comparison assumptions
STA	Do required paths/limits pass across scenarios?	Setup/hold, recovery/removal, slew/load
DRC	Does geometry meet checked process rules?	Width, spacing, vias, density
LVS	Does extracted circuitry match its reference?	Open/short; device/parameter mismatch
Power / EM / IR	Can supply and conductors meet electrical/reliability limits?	Droop; excessive current density
DFT / ATPG	Are test access and modeled fault coverage adequate?	Untestable state; wrong test constraints

LVS can use device-level or hierarchical comparison with suitable cell/macro views. It is not RTL simulation. Passing means agreement with the reference under the extraction/comparison setup, not proof that the reference specification is right. DRC evidence likewise depends on suitable, complete rules. Waivers need explicit justification and acceptance.

Contrast question

Can a design pass DRC but fail LVS? Yes: geometrically legal wires can connect the wrong circuit. Can it pass both and fail timing? Yes: correct legal connections can still be too slow or too fast for setup/hold.

App practice: Predict which check detects each defect

1. Open **Signoff (08)** and choose **Clean example**. Record DRC PASS and LVS PASS for the toy case.
2. Before clicking **Spacing violation**, predict both results. Then observe DRC FAIL / LVS PASS: the 5-unit gap violates a 10-unit rule while the expected connections remain.
3. Choose **Broken connection**. Observe DRC PASS / LVS FAIL: the spacing example is acceptable but the reference connection is open.
4. Return to **Clean example**, then open Timing and choose After routing. Explain how passing the first two checks can coexist with failing setup.

Explain. Geometry, circuit agreement, and timing answer different questions. These defect buttons select simplified teaching cases; they do not run extraction, a DRC rule deck, or an LVS engine. Passing their toy checks is not physical signoff.

19 Closure: repairs must survive the checks

Lecture window: 31–32:30 minutes. *Diagnose before changing the implementation.*

Plain-language starting point. When a check fails, identify the cause, change the design, and check again. A repair that helps one path can worsen another or require more space and power.

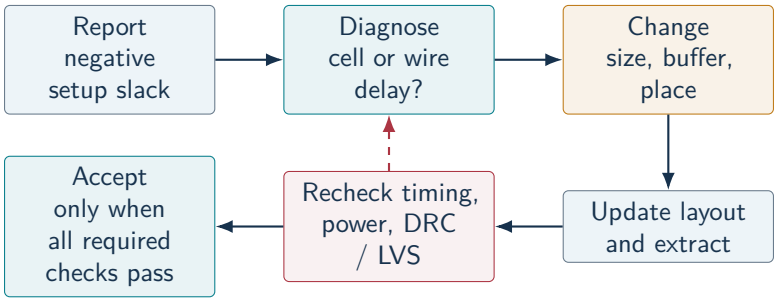


Figure 23: Diagnose, repair, update extraction, and recheck the current implementation.

For −0.05 ns setup slack, inspect startpoint/endpoint, mode/corner, constraints, clock edges, and delay contributions. Is the limiting cause cell delay, a long/loaded net, poor slew, or a mistaken timing contract? Choose a repair based on the cause.

Observation	Candidate repair	Consequences to recheck
Wire-dominated delay	Move cells; appropriate buffering	Load, routing, hold, power
Cell-dominated delay	Resize / restructure	Upstream capacitance, area, equivalence
Short-path hold failure	Add suitable data delay	Setup at all corners, slew, routing
Local congestion	Spread cells / revise floorplan	Area, wirelength, supply routing
Supply droop	Improve power connectivity	EM, space, activity assumptions

A setup speedup can worsen hold; a hold repair can worsen setup. A buffer may speed a long RC net despite intrinsic delay, but that benefit is context-dependent. Larger cells may raise load or congestion. An engineering change order (ECO) requires checks appropriate to the actual change.

Keep RTL/netlist/layout revisions, constraints, library/PDK versions, parasitics, tool configuration, and reports traceable. Timing on one revision plus LVS on another does not sign off either implementation.

App practice: Repair the model and recheck both constraints

Controls for this activity. If a step changes a slider, click **Show equations and sliders** in Guided view first.

1. In **Timing (04)**, choose **After routing**. Record setup −0.05 ns and hold −0.04 ns at the original 2.00 ns period.
2. Move **Added hold-buffer delay** to 0.06 ns. Hold improves to +0.02 ns, but setup worsens to −0.11 ns. Diagnose the tradeoff before trying another change.
3. Keep period 2.00 ns, skew 0, and buffer delay 0.06 ns; reduce **Wire delay (max)** to 0.45 ns. Check setup +0.04 ns and hold +0.02 ns.
4. Record before/after parameters and list the real obligations: implement the repair, update extraction, and repeat affected timing, geometry/connectivity, equivalence, and power/reliability checks.

Explain. The two positive cards demonstrate a solution to this scalar model. The wire slider stands for a hypothetical implementation improvement; no route was shortened, no buffer was physically inserted, and no other signoff scenario was verified by moving it.

20 Tapeout is the handoff; manufacturing creates the chip

Lecture window: 32:30–33:30 minutes. *Finish the physical-chip story.*

Plain-language starting point. Tapeout hands approved layout information to manufacturing. The factory must still fabricate and test the devices, package them, and validate their operation.

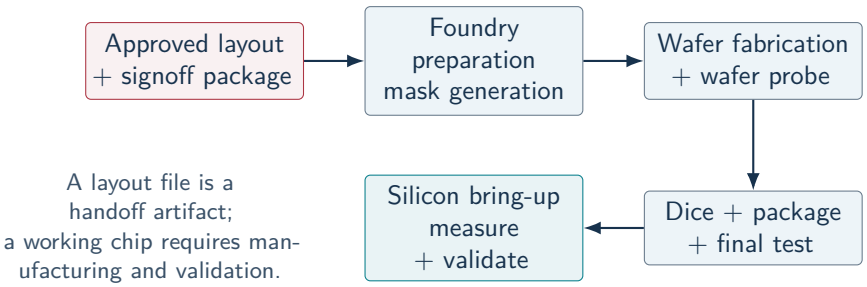


Figure 24: Beyond layout: fabrication, packaging, test, and silicon validation.

Tapeout transfers the approved layout and documentation for manufacturing preparation. Foundry work includes mask-related preparation. Repeated fabrication steps create many die on a wafer; wafer probe helps identify defective die. Dicing, packaging, and final test produce packaged devices. Specific assembly/test flows vary.

Silicon bring-up measures clocks, supplies, reset, interfaces, expected behavior, performance, power, and the operating envelope. Real devices can expose issues not represented in available models. Yield and reliability also matter. A functioning lab sample does not establish product qualification.

For the counter, functional evidence explains why it counts; timing establishes the modeled clock/interface contract; physical checks address geometry/connectivity; manufacturing test and silicon validation address actual devices.

App practice: Audit what is still missing before handoff

1. In **Signoff (08)**, select Clean example and read **Other checks still matter**. List which listed evidence categories are absent from the app.
2. In Gates & mapping, download the HDL source and circuit SVG. Explain why those files are useful learning artifacts but cannot be submitted as fabrication-ready layout.
3. Trace Figure 24 from approved layout through fabrication, packaging, test, and bring-up. For each stage, name one activity that a local digital simulator cannot perform.

Explain. The app ends at teaching representations and functional/model experiments. It generates no approved GDSII/OASIS, masks, manufacturing test result, packaged device, or silicon measurement. Tapeout and bring-up require evidence and work beyond this companion.

21 Closing exercise and retrieval recap

Lecture window: 33:30–36 minutes. *Use the flow to explain a failure.*

Plain-language starting point. Use the counter story to diagnose a failure. Correct counting with a late arrival is a timing issue; a broken metal connection belongs to physical connectivity checking.

Prediction question 3: diagnose the evidence

A design passes RTL simulation and pre-route setup timing. After routing it has −0.05 ns setup slack, passes DRC, and passes LVS. Is it ready for its 2 ns clock? What should we inspect first and repeat after repair?

Expected answer. It does not meet that timing target. Inspect the failing path, constraints, clock relationship, and extracted cell/wire delays. Repair the cause; repeat affected extraction, setup/hold/electrical checks, physical checks, and applicable power/reliability analysis. Revisit equivalence/functional evidence if logic changed.

Ask for four answers without reading the map:

1. What does synthesis add? **Target-library cells and connectivity.**
2. What do placement/routing add? **Locations and interconnect geometry.**
3. Why does slower frequency help setup but not ordinary hold? **Setup uses the next edge; same-edge hold has no period term.**
4. DRC versus LVS? **Geometry compliance versus extracted-circuit agreement.**

Student-facing close. "We began with a rule for updating four bits. Synthesis selected cells, physical design arranged and connected them, and separate checks asked whether the implementation behaved correctly, met timing, matched its circuit, and obeyed physical rules. Keep these questions connected, while recognizing that each needs its own evidence."

Use the remaining interview time for questions about assumptions, tradeoffs, or teaching choices. The appendices support preparation and later material development; they are outside the 40-minute core.

App practice: Explain a failure across the flow

1. Select and restart the ordinary counter in **RTL & behavior (02)**. Step once and identify functional evidence that count becomes 1.
2. In Timing, select After routing and record setup −0.05 ns. In Signoff, choose Clean example and observe the two passing toy checks. Use these separate illustrations to explain the diagnosis in Prediction question 3.
3. Without looking at the map, answer the four retrieval questions above. For each answer, identify one app action that makes it visible.
4. Return to Design intent and explain one feedback loop: a timing problem can require an implementation change, followed by updated evidence.

Explain. The screens are independent examples, not a shared physically implemented design. Their combined story illustrates why functional correctness, timing, and physical checks must each be established for the *same* release candidate in a real flow.

A Preparation questions and concise answers

Lecture window: Preparation / Q&A. *Deepen understanding without extending the core lecture.*

Plain-language starting point. Use these answers after you understand the first-pass route. A useful answer names the question, the mechanism, and the evidence needed to check it.

1. **Why RTL rather than every transistor?** It expresses behavior/state at a manageable level. Synthesis and libraries reuse characterized implementations. Full-custom design remains important for cells, memories, and analog blocks. FPGA flows instead map into a prefabricated programmable fabric.

2. **Does synthesis guarantee correctness?** It aims to preserve synthesizable semantics under assumptions. Equivalence provides separate preservation evidence; neither establishes that the initial RTL meets an unstated requirement.
3. **Can synthesis infer a latch?** Yes, incomplete combinational assignments can require storage. A clocked flip-flop process holding state is different.
4. **Standard cell versus macro?** A standard cell fits regular row/site placement with compatible views. A macro is a larger predesigned block, often SRAM, with its own dimensions and constraints.
5. **Why not optimize only area?** Small gates may have insufficient drive or slow paths; dense placement may be unroutable. Timing, power, test, physical, and cost constraints interact.
6. **Why can more drive strength hurt?** A larger cell can increase its input capacitance, upstream delay, power, and area. Analyze the network rather than one cell.
7. **Is zero skew always the objective?** Manageable skew is useful; intentional useful skew can improve selected paths. Setup and hold across all scenarios constrain the choice.
8. **Different clock frequencies?** Define genuine/generated relationships and correct edge pairing. Asynchronous transfers require appropriate protocols and CDC verification; the simple equations alone are insufficient.
9. **Can every CDC be a false path?** No. Constraint exceptions and architectural synchronization address different questions; required synchronizer/bus-skew/other checks must remain valid.
10. **Why temperature corners?** Models vary with conditions; temperature inversion prevents a universal "hot is slow" rule. Check the characterized scenarios required for the design.
11. **What is an unconstrained path?** A required relationship/budget is missing. Its absence from violating-path reports is not evidence that it passes.
12. **How can a buffer speed a net?** It segments an RC/load problem and changes slew, possibly outweighing intrinsic delay. On a short path, added delay may instead help hold.
13. **Why scan affects layout?** Flops, wiring, ports, and test clocks consume resources; chain order affects wirelength. Plan test before final closure.
14. **Is DRC-clean always manufacturable?** It is evidence against the selected deck. Foundry acceptance can need complete views, other checks, correct versions, and approved waivers.
15. **Does LVS prove counting?** It compares against the reference circuit. A wrong reference can still pass LVS.
16. **Logical versus physical ECO?** One changes circuit structure; another adjusts implementation, sometimes including cells. Recheck obligations depend on what changed.
17. **A follow-up lab?** Pin a supported public platform/toolchain, provide RTL/constraints and a self-checking testbench, and compare saved implementation reports. Require explanations grounded in paths and metrics.
18. **Connection to software/security expertise?** Interface contracts, explicit assumptions, traceability, and diagnosis are useful bridges. Teach the required ASIC content directly at the stated abstraction level.

App practice: Rehearse explanations with evidence

1. Choose one question about synthesis, one about timing, and one about signoff. Give a 30-second answer before opening the corresponding app lesson.
2. Demonstrate the relevant action: inspect an adder/gate, change capture skew, or select a spacing/open defect. Explain what the observation supports and what it leaves untested.
3. Open **Teaching notes** with N. Use its Demonstrate, Explain, and Bridge prompts to rehearse the transition to the next slide. Keep a timed record and return to the document for details outside the app.

Expected result. A concise explanation connected to an observable example, with appropriate scope. CDC protocols, PVT corners, scan, real power integrity, and production acceptance remain discussion/document topics rather than simulated features of this app.

B Student practice with solutions

Lecture window: Optional homework / rehearsal. *Short tasks reveal conceptual errors.*

Plain-language starting point. Predict before operating the app. Then compare your prediction with the observation and explain any difference. Keep setup and hold separate.

1. State evolution. After reset, $q = 0$. Apply enables $1, 1, 0, 1$ on successive edges, reset low. Then predict $q = 15$, $en = 1$.

Solution. $1, 2, 2, 3$; then 0 from modulo-16 wraparound.

2. Setup with skew. $T = 1.80$ ns, $t_{cq,max} = 0.15$ ns, $t_{data,max} = 1.10$ ns, $t_{setup} = 0.15$ ns, $U_{setup} = 0.30$ ns, $s = -0.04$ ns. Find slack. The fixed cell and wire values match the app's pre-route baseline.

Solution. $1.80 - 0.04 - 0.15 - 1.10 - 0.15 - 0.30 = \boxed{+0.06 \text{ ns}}$. Negative skew reduces setup budget.

3. Hold with skew. $t_{cq,min} = 0.06$ ns, $t_{data,min} = 0.04$ ns, $t_{hold} = 0.12$ ns, $U_{hold} = 0.02$ ns, $s = +0.02$ ns. Find slack and required additional modeled delay. The minimum data delay is $0.03 + 0.01$ ns in the app.

Solution. $0.06 + 0.04 - 0.02 - 0.12 - 0.02 = \boxed{-0.06 \text{ ns}}$. At least 0.06 ns more minimum delay reaches zero in this model; real repair must pass all scenarios and required margins.

4. Utilization. For 30,000 μm^2 standard cells in a macro-free block, estimate core area at 0.60 and 0.75 utilization. Is 0.75 necessarily better?

Solution. 50,000 and 40,000 μm^2. Smaller area can worsen congestion and repair space; inspect feasibility.

5. Classify issues. A too-narrow wire; an unintended short; late data; incorrect reset priority; excessive voltage droop.

Solution. DRC; LVS and routing-short checks; STA; functional verification; IR analysis. Some issues involve several checks.

6. Critique a claim. "We generated GDS and passed RTL simulation, so the chip is production-ready." Name missing evidence categories.

Solution. Final extracted timing, approved complete physical checks, supply/reliability analysis, test access/fault coverage, appropriate equivalence, and manufacturing/silicon validation as required. A layout file is an intermediate handoff artifact.

App practice: Check the written exercises in the app

1. For Exercise 1, restart the ordinary counter, keep Reset off, and apply Enable values 1, 1, 0, 1 with a step after each. Record 1, 2, 2, 3; export the trace. Verify wraparound in a separate restart-and-16-steps run.
2. For Exercise 2, open Timing and click Before routing. Set Clock period to 1.80 ns and Capture clock skew to −0.04 ns. Compare setup +0.06 ns with your paper calculation.
3. For Exercise 3, reload Before routing and set skew to +0.02 ns: hold is −0.06 ns. Set Added hold-buffer delay to 0.06 ns: hold reaches 0.00 ns; 0.07 ns gives +0.01 ns. Check setup after each change.
4. For Exercise 4, use the floorplan slider to illustrate 60% versus 75% utilization, but calculate the area on paper. For Exercises 5–6, use the spacing/open cases as examples and list the other issue/evidence categories that they do not exercise.

Submission. Include predictions, recorded values, a short explanation, and the relevant CSV or calculation. Distinguish measured coordinates in the toy model, functional simulation results, illustrative arithmetic, and missing physical-tool evidence.

C A reproducible lab extension and report-reading guide

Lecture window: Optional after the interview. *Separate the implemented app from a proposed physical implementation lab.*

Plain-language starting point. A full physical-design lab would run additional tools against technology data. The current app is useful for synthesis and functional experiments plus clearly marked physical teaching models.

ORFS integrates Yosys, OpenROAD, and KLayout for an educational RTL-to-GDS route [2]. Choose a supported public platform, pin tools/platform versions, confirm licenses/RTL support, and follow current official setup/design configuration instructions.

1. **Behavior.** Use a self-checking testbench for reset priority, hold, increments, and wraparound. Record the reference rule and environment assumptions.
2. **Contract.** Provide clock, input/output min/max timing, electrical conditions, and scenarios; confirm clock/port coverage.
3. **Synthesis.** Inspect inferred state, mapped cells, warnings, and reports. Use supported equivalence tooling if available; a synthesis log is not equivalence evidence.
4. **Implementation.** Inspect floorplan, cells, clock tree, routes, congestion, and stage-specific timing; save the supporting artifacts.
5. **Comparison.** Compare estimated pre-route and extracted timing. Inspect the reported paths before attributing a change to wire delay.
6. **Checks.** Use matching design data for the available checks. Record which production signoff checks remain unavailable.

Read one timing report. Identify startpoint/endpoint; group, mode/corner; launch/capture edges; clock latencies; cell/net contributions; transition/load; arrival, required time, and slack. Interpret the clock relationship before judging the sign.

Compare variants. Keep behavior, tool/library versions, scenarios, workload, and metric definitions consistent. Distinguish cell area from core/die area. Compare complete violations and coverage, not only WNS. Architectural changes also require latency/throughput and verification comparisons.

Suggested lab deliverable. One page with requirements, an actual implementation diagram, a timing-path explanation, defined PPA metrics, and a proposed repair with tradeoffs. Distinguish tool outputs,

estimates, and qualitative reasoning.

The included RTL/SDC are teaching examples. The app's synthesis and functional simulation are implemented and checked separately from document compilation. The proposed ORFS physical implementation lab has not been executed for this tutorial. TikZ diagrams and app physical views are original teaching schematics rather than physical-tool outputs.

App practice: Build an app experiment before a physical-tool lab

1. In Gates & mapping, restore the counter and synthesize it. Record the source, Top module, tool version, and cell counts shown in Synthesis report. Save the HDL and SVG.
2. Select increment-by-two and repeat with the same stimulus assumptions. Save a trace and explain the deliberate change in behavior; cell-count differences alone are not a like-for-like PPA comparison.
3. Write a manifest separating real synthesis/simulation artifacts from hypothetical timing/physical cases. The app's `package-lock.json` pins its dependencies; its README describes startup and checks.
4. If doing the proposed ORFS lab, obtain actual matched netlist, library/PDK views, constraints, layout, parasitics, and reports outside this app. Then read the real timing path using the report guide above.

Explain. The app supplies a reproducible first experiment. Its generic synthesis log is not a foundry timing report, its timing cards are not WNS/TNS over a design, and its SVG/physical illustration is not an implemented layout. Keep the two kinds of lab evidence distinct.

D Reusable visual catalog

Lecture window: Slide and handout preparation. *Each diagram has editable standalone TikZ source.*

Plain-language starting point. Choose a figure that answers one question. Explain the main relationship before reading its smaller technical labels.

No.	Filename stem	Teaching use
1	`01-flow`	Opening/closing flow map and feedback.
2	`02-abstraction`	Behavior, connectivity, geometry.
3	`03-counter-datapath`	State update, enable, reset priority.
4	`04-counter-waveform`	Edge-based update and hold.
5	`05-verification`	Simulation, properties, equivalence.
6	`06-mapping`	Library-based synthesis.
7	`07-ppa`	Sizing and PPA tradeoffs.
8	`08-artifacts`	RTL/SDC/Liberty/LEF/DEF/SPEF/layout.
9	`09-timing-path`	Clock latencies and skew.
10	`10-setup`	Latest arrival and setup slack.
11	`11-hold`	Earliest arrival and hold repair.
12	`12-constraints`	Timing/electrical interface budgets.
13	`13-scan`	Shift/capture manufacturing test.
14	`14-floorplan`	Die/core, rows, macro halo, I/O.
15	`15-placement`	Global placement versus legalization.
16	`16-clock-tree`	Buffered delivery, latency, skew.
17	`17-routing`	Metal connections and vias.
18	`18-parasitics`	Wire RC and coupling.
19	`19-power-grid`	Supply delivery, IR drop, EM.
20	`20-drc-lvs`	Geometry versus circuit agreement.
21	`21-closure`	Diagnose, repair, extract, recheck.

No.	Filename stem	Teaching use
22	`22-tapeout`	Manufacturing, packaging, bring-up.

The main source is self-contained and uses `\usetikzlibrary`. Standalone sources in `figures/` reproduce the same definitions/styles. Vector PDF is sharp at slide scale; PNG is convenient in PowerPoint. Preserve the assumptions when adapting figures. The tutorial captions remain the authoritative explanation.

Three additional diagrams introduce timing before its full notation: `beginner-01-data-journey` shows the source, calculation, wire, and receiving register; `beginner-02-setup-deadline` compares arrival with a deadline; `beginner-03-hold-window` shows the short protection window after sampling. Each has the same editable TikZ, vector PDF, and PNG exports. Start with these, then use diagrams 9–11 when the detailed timing questions arise.

For Beamer, use `\includegraphics` with exported PDF, or copy a diagram macro plus common styles and add overlays. Check projection legibility. The core route deliberately selects fewer visuals than the full library; all 25 do not need their own slide.

App practice: Prepare a static slide and a live experiment

1. Open the lesson matching your slide, then **Teaching notes**. Use **Download figure PDF** for the original TikZ visual and consult the corresponding filename in this catalog.
2. In RTL & behavior, restore and restart the counter, step twice, and select the adder. Fit/zoom the schematic, then use **Download circuit SVG** to save that functional circuit state. Export its waveform CSV if useful.
3. Put the original figure on the slide; annotate the action you will try and its expected result. During rehearsal, press P for presentation mode and perform just that short action.
4. Check projection legibility and retain a static figure as a fallback. Restore the prepared source and starting inputs before the next rehearsal.

Explain. The PDF is an original reusable teaching diagram; the SVG captures an actual compiled circuit visualization. Neither is a foundry layout screenshot. The document's standalone TikZ sources remain editable independently of the app.

E Glossary

Lecture window: Preparation reference. *Keep terms consistent.*

Plain-language starting point. Use this glossary as a translation aid. Associate each new term with a circuit job or app observation before memorizing its formal definition.

Term	Meaning
ASIC	Application-specific integrated circuit.
RTL	Register-transfer level: state and update/combinational rules.
EDA	Electronic design automation.
Netlist	Instances and pin-to-net connectivity.
Standard cell	Characterized logic/state cell for regular placement.
PDK	Process design kit: technology models, layers, rules, data.
PPA	Power, performance, area.
STA	Static timing analysis of constrained paths.
Setup / hold	Stability requirements before / after capture.
Slack	Margin relative to a timing requirement.
CTS / CDC	Clock tree synthesis / clock-domain crossing.
DFT / ATPG	Design for test / automatic test-pattern generation.
DRC / LVS	Design rule checking / layout versus schematic.
RC / SPEF	Resistance-capacitance / parasitic exchange format.
IR / EM	Supply voltage drop / electromigration.
ECO	Engineering change order.
Tapeout	Approved layout/design handoff for manufacturing.

App practice: Attach vocabulary to observable examples

1. Visit RTL & behavior, Gates & mapping, and Floorplan & place. Define RTL, netlist, and placement using what each screen shows; identify which physical view is conceptual.
2. In Timing, use Before routing and name arrival, required time, setup, hold, and slack. Explain their units and why the waveform's event index is not time in ns.
3. In Signoff, show one spacing failure and one open connection. Define DRC and LVS, then name a term in this glossary that the app does not implement, such as ATPG, SPEF extraction, or EM analysis.

Expected result. A term, a concrete example, and an appropriate evidence boundary for each answer. Add these to a personal study sheet rather than memorizing file extensions without their roles.

Source notes and further reading

Primary documentation was consulted October 6, 2026 for tool roles, interfaces, and flow examples. Practice instructions were checked against the supplied app's source, examples, controls, and model tests. Diagrams, toy requirements, calculations, and exercises were created for this tutorial. Individual capabilities and foundry requirements are version/platform-specific; the snippets do not reproduce a complete toolchain.

App practice: Distinguish a tool role from a demonstrated result

1. In Gates & mapping, record the Yosys version and inspect the synthesis report. Follow the tool documentation below to check what its mapping step does.
2. Compare the DigitalJS and conversion documentation with the app's functional circuit and trace. State which observations come from the implemented simulator.
3. Choose a physical capability, such as CTS or extraction, and identify the corresponding official source. Explain what actual input/model/report evidence would be needed beyond the app's illustration.

Explain. Documentation establishes tool interfaces and intended roles. The supplied app's checks establish its tested examples. Neither independently proves educational effectiveness or that a physical chip has passed signoff.

References

[1] Carnegie Mellon University, Eberly Center. *Lectures: structuring content, examples, and interaction.* https://www.cmu.edu/teaching/designteach/teach/instructionalstrategies/lectures.html.

[2] OpenROAD Project, *OpenROAD Flow Scripts User Guide.* Integrated RTL-to-GDS example and tool roles. https://openroad-flow-scripts.readthedocs.io/en/latest/user/UserGuide.html

[3] YosysHQ, *Technology mapping.* Abstract structures and target-library mapping. https://yosyshq.readthedocs.io/projects/yosys/en/stable/using_yosys/synthesis/techmap_synth.html

[4] OpenROAD / Parallax, *Parallax Static Timing Analyzer.* Timing-analysis file and clock interfaces. https://openroad.readthedocs.io/en/latest/main/src/sta/README.html

[5] OpenROAD Project, *Clock Tree Synthesis.* https://openroad.readthedocs.io/en/latest/main/src/cts/README.html

[6] OpenROAD Project, *Parasitics Extraction.* RC models and SPEF. https://openroad.readthedocs.io/en/latest/main/src/rcx/README.html

[7] OpenROAD Project, *DFT: Design for Testing.* Scan replacement and chain construction. https://openroad.readthedocs.io/en/latest/main/src/dft/README.html

[8] KLayout, *Design Rule Checking: Basic Features.* https://www.klayout.de/doc/manual/drc_basic.html

[9] KLayout, *Layout vs. Schematic (LVS): Overview.* https://www.klayout.de/doc/manual/lvs_overview.html

[10] SkyWater, *SKY130 PDK documentation*. Process/library data and release status.
https://skywater-pdk.readthedocs.io/en/main/

[11] AMD, *UltraFast Design Methodology Guide, UG949: Multi-Bit CDC*. FPGA-oriented coherence guidance; device-specific details do not transfer to ASICs.
https://docs.amd.com/r/en-US/ug949-vivado-design-methodology/Multi-Bit-CDC

[12] YoWASP, *WebAssembly-based packages for Yosys and related tools*. Runtime packaging used for local synthesis.
https://yowasp.org/

[13] Marek Materzok, *DigitalJS: a Visual Verilog Simulator for Teaching*, CSERC '19, published 2020. Implementation documentation and teaching publication.
https://github.com/tilk/digitaljs
https://doi.org/10.1145/3375258.3375272

[14] `yosys2digitaljs`, *Export netlists from Yosys to DigitalJS*. Conversion between the synthesis and simulator representations.
https://github.com/tilk/yosys2digitaljs